

Konzeption und Evaluation eines
Multi-Agenten-Systems zur automatisierten
Migration von imperativer LO-VC Logik zu
deklarativer AVC Syntax am Beispiel der msg
systems ag

Torben Dörrer

8. August 2026

Inhaltsverzeichnis

Abkürzungsverzeichnis	iv
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Forschungsfragen	2
1.3 Methodisches Vorgehen und Forschungsdesign	3
1.4 Abgrenzung der Arbeit	4
1.5 Aufbau der Arbeit	4
2 Theoretische Grundlagen	5
2.1 Domänenspezifische Grundlagen: SAP Variantenkonfiguration	5
2.1.1 Einführung in die logische Architektur der Variantenkonfiguration	5
2.1.2 Die klassische Variantenkonfiguration (LO-VC) im Kontext der Systemtransformation	7
2.1.3 Die moderne Architektur (AVC) und das Constraint Paradigma	8
2.2 Generative künstliche Intelligenz im Software Engineering . . .	9
2.2.1 Large Language Models für Quellcode	9
2.2.2 Limitationen monolithischer Ansätze	12
2.3 Multi-Agenten-Systeme(MAS)	14
2.3.1 Definition und kognitive Architekturen	14
2.3.2 Kollaborative Agenten-Muster	17
2.3.3 Entwicklungs-Frameworks für MAS	19
3 Related Work	22
4 Durchführung und Auswertung des Experteninterviews	26
4.1 Zielsetzung und Erkenntnisinteresse	26
4.2 Methodisches Forschungsdesign	27
4.3 Datenerhebung und Auswertungsprozess	28
4.4 Ergebnisse der Befragung	29

4.4.1	Kognitive Altlasten und historischer Code	29
4.4.2	Paradigmenwechsel und syntaktisches Umdenken	29
4.4.3	Fragmentierung manueller Prozessschritte	30
4.4.4	Limitierungen bestehender Standard-Tools	30
4.4.5	Manueller Aufwand und Fehleranfälligkeit im Testprozess	30
4.4.6	Zeitliche Metriken und Identifikation von Flaschenhälsen	31
4.5	Implikationen für die Systemarchitektur (Ideales Zielbild)	31
4.5.1	Funktionale Anforderungen aus der Expertenperspektive	31
4.5.2	Architektonische Paradigmen und Beratungsansatz	32
4.5.3	Hypothese der Multi-Agenten-Rollen	32
4.5.4	Quantitative Zielvorgaben für die Systemvalidierung	33
4.5.5	Überleitung zum methodischen Realitätsabgleich	33
5	Durchführung und Auswertung des praktischen Selbstversuchs	34
5.1	Technologischer Realitätsabgleich und empirische Scope-Definition	34
5.1.1	Evaluation der funktionalen Praxisanforderungen	35
5.1.2	Evaluation der architektonischen Paradigmen	36
5.1.3	Synthese des finalen System-Scopes	36
5.2	Wissenschaftliche Einordnung und methodisches Analyseprotokoll	37
5.3	Datenbasis und Sampling-Strategie	38
5.4	Ergebnisse des Selbstversuchs: Ableitung der kognitiven Architektur	38
5.4.1	Ableitung der V1-Architektur und System-Prompts	40
6	Systemdesign und Prototypenentwicklung	42
6.1	Technologieauswahl und Infrastruktur	42
6.1.1	Orchestrierungs-Framework: CrewAI	42
6.1.2	Infrastruktur und Modellintegration: SAP AI Core und liteLLM	43
6.2	Iterative Entwicklung des Prototyps (Design Science Research)	44
6.2.1	Zyklus 1: Limitierungen einer rein agentenbasierten Architektur	44
6.2.2	Zyklus 2: Code-Refaktorisierung, Wissensbasis und strukturelle Halluzinationen	45
6.2.3	Zyklus 3: Regelbasierte Vorverarbeitung und Domänenunabhängigkeit	46
6.2.4	Zyklus 4: Die finale neuro-symbolische Architektur und funktionale Abgrenzung	47
	Anhang Anhang	53

<i>INHALTSVERZEICHNIS</i>	iii
Anhang Datenauswertung der Experteninterviews	54

Abkürzungsverzeichnis

KMAT	konfigurierbare Materialstamm
Super BOM	konfigurierbare Maximalstückliste
AVC	Advanced Variant Configuration
LO-VC	Logistics Variant Configuration
KI	künstliche Intelligenz
ML	Machine Learning
CSP	Constraint Satisfaction Problem
LLMs	Large Language Models
ICL	In-Context Learning
MAS	Multi-Agenten Systeme
DSR	Design Science Research

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Im Zuge der digitalen Transformation und der Abkündigung älterer ERP-Generationen stehen produzierende Unternehmen weltweit vor der komplexen Herausforderung, ihre IT-Landschaften auf das moderne System SAP S/4HANA zu migrieren. Ein hochgradig kritischer Teilbereich dieser Transition betrifft die Variantenkonfiguration: Die historisch gewachsene, imperative Logik der klassischen *Logistics Variant Configuration* (LO-VC) muss in die moderne, deklarative Syntax der *Advanced Variant Configuration* (AVC) überführt werden.

In der industriellen Praxis – insbesondere im Beratungsumfeld der *msg systems ag* – zeigt sich, dass diese Syntax-Migration einen massiven zeitlichen und finanziellen Flaschenhals darstellt. Da direkte, maschinelle 1:1-Übersetzer aufgrund der grundlegenden Paradigmenwechsel zwischen imperativen und deklarativen Regelwerken fehlen, erfolgt die Überführung oftmals in manueller, extrem fehleranfälliger Fleißarbeit. Erste Versuche der Industrie, diesen Prozess durch moderne generative *künstliche Intelligenz* (KI) zu automatisieren, blieben bislang hinter den Erwartungen zurück. Wie Voruntersuchungen zeigen, scheitern monolithische Single-Prompt-Ansätze (wie die einfache Abfrage über Werkzeuge wie ChatGPT) an der strukturellen Komplexität und den strikten Syntaxvorgaben der SAP-Constraint-Netzwerke. Das Sprachmodell verliert bei langen Logikketten den Kontext und neigt zu nicht-kompilierbaren Halluzinationen.

Um diesen Flaschenhals informationstechnisch aufzulösen, rückt das Paradigma der *Multi-Agenten Systeme* (MAS) in den Fokus der angewandten Informatik. Die Hypothese lautet, dass eine Aufteilung von Teilbereichen der komplexen Übersetzungsaufgabe in ein orchestriertes Kollektiv spezialisierter

KI-Agenten die Code-Qualität und die Arbeitsgeschwindigkeit bei der Migration drastisch erhöhen kann, sofern die Ergebnisse der KI von Entwicklern in einem iterativen, agentischen Workflow validiert und konsolidiert werden.

1.2 Zielsetzung und Forschungsfragen

Das primäre Ziel dieser Masterarbeit ist die theoriegeleitete Konzeption und praktische Implementierung einer Multi-Agenten-Architektur zur automatisierten Code-Migration von LO-VC nach AVC. In Kooperation mit der *msg systems ag* soll nicht nur die theoretische Eignung von Agenten-Frameworks evaluiert, sondern ein lauffähiger, cloudbasierter Prototyp auf der *SAP Business Technology Platform* (BTP) entwickelt und bewertet werden.

Um dieses Ziel zu erreichen, wird die Arbeit von folgender Hauptforschungsfrage geleitet: **In welchem Umfang kann ein Multi-Agenten-System die Effizienz und Qualität bei der Migration von imperativer LO-VC-Logik zu deklarativer AVC-Syntax im Vergleich zum manuellen Vorgehen steigern?**

Zur strukturierten Beantwortung dieser Hauptfrage werden vier dedizierte Unter-Forschungsfragen (FF) definiert:

- **FF1 (Baseline):** Welche manuellen Prozessschritte sind bei der Migration zwingend erforderlich und wie hoch ist der zeitliche Aufwand pro Logik-Objekt?
- **FF2 (Werkzeug-Wahl):** Anhand welcher technischen und konzeptionellen Kriterien lässt sich das optimale Multi-Agenten-Framework (z. B. CrewAI vs. AutoGen vs. LangChain) für das Einsatzumfeld der automatisierten Code-Migration identifizieren und rechtfertigen?
- **FF3 (Architektur):** An welchen Stellen der Migration bietet sich der Einsatz eines MAS an und wie lässt sich der herausgearbeitete Scope systematisch in spezialisierte agentische Rollen überführen?
- **FF4 (Impact):** Welche Einsparung (in Bearbeitungszeit) und welche Qualitätsrate (Syntax-Korrektheit) erzielt der agentische Ansatz im Vergleich zur manuellen Baseline?

1.3 Methodisches Vorgehen und Forschungsdesign

Um eine valide, praxistaugliche und wissenschaftlich fundierte Multi-Agenten-Architektur für die Migration von LO-VC- nach AVC-Code zu konzipieren, folgt diese Arbeit dem Paradigma des *Design Science Research* (DSR) nach Hevner et al. (2004). Ziel dieses gestaltungsorientierten Forschungsansatzes ist die Konstruktion und Evaluation eines innovativen, zweckmäßigen IT-Artefakts – in diesem Fall des Modells und der nachfolgenden Instanziierung einer Multi-Agenten-System-Architektur –, um ein komplexes, unstrukturiertes Problem (*Wicked Problem*) in einem realen technologischen Umfeld nachhaltig zu lösen.

Entsprechend dem IS-Forschungsmethoden-Framework von Hevner et al. (2004) gliedert sich das Forschungsdesign dieser Arbeit in zwei komplementäre Kernphasen, welche die Balance zwischen praktischer Relevanz (*Relevance*) und methodischer Strenge (*Rigor*) sichern:

1. **Explorative Phase zur Problem- und Scope-Definition (Umfeld & Relevanz):** Den Ausgangspunkt bildet die präzise Erfassung des Problemraums und des *Business Needs* innerhalb des organisationalen und technologischen Umfelds (*Environment*). Hierzu wird eine explorative Fallstudie nach den Richtlinien von Runeson und Höst (2009) vorgeschaltet. Um Verzerrungen zu minimieren und den konkreten funktionalen Scope des MAS empirisch abzugrenzen, erfolgt eine Datentriangulation aus drei Quellen:
 - *Archivdokumentenanalyse:* Systematische Untersuchung bestehender SAP-Dokumentationen, Syntaxvorgaben und Altsystemstrukturen zur Definition der technologischen Rahmenbedingungen.
 - *Qualitative Experteninterviews:* Erhebung von implizitem Praxiswissen (*Tacit Knowledge*) von Domänenexperten bezüglich manueller Migrationsaufwände (Baseline) sowie realer Fallstricke beim Einsatz generativer KI.
 - *Selbstversuch:* Durchführung einer eigenen, manuellen Beispielmigration zur Identifikation inhärenter syntaktischer Transformationsbarrieren im LO-VC-Code.

Die Synthese dieser Erkenntnisse fundiert den Problemraum wissenschaftlich und führt direkt zur Ableitung einer ersten fundierten Version 1 (V1) der Systemarchitektur.

2. **Iterativer DSR-Gestaltungs- und Evaluationszyklus (Konstruktion & Strenge):** Aufbauend auf der Architekturversion V1 setzt der eigentliche Kernprozess des Design Science Research ein. Im Sinne des von Hevner et al. (2004) beschriebenen *Generate/Test Cycle* (Suchprozess) wird das Artefakt im Zuge der softwaretechnischen Umsetzung schrittweise und iterativ weiterentwickelt (V1 zu V2 bis zur finalen Architektur). Jede Iteration greift dabei auf theoretische Grundlagen aus der Wissensbasis (*Knowledge Base*) zurück – wie etwa etablierte Architekturmuster für Multi-Agenten-Systeme sowie Prinzipien des Compilerbaus –, um die methodische Strenge (*Rigor*) zu gewährleisten. Abschließend wird das fertige Artefakt einer rigorosen Evaluation (z. B. durch funktionales Testing mit realitätsnahen Migrationsszenarien) unterzogen, um dessen Nützlichkeit (*Utility*) empirisch nachzuweisen.

Durch diese methodische Verknüpfung wird sichergestellt, dass die vorgeschlagene MAS-Architektur nicht nur auf strengen wissenschaftlichen Methoden beruht, sondern auch einen messbaren, praktischen Beitrag für die automatisierte Code-Transformation in der industriellen SAP-Praxis leistet.

1.4 Abgrenzung der Arbeit

1.5 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in sechs aufeinander aufbauende Kapitel. Nach der Einleitung in **Kapitel 1** schafft **Kapitel 2** das theoretische Fundament, indem es die Domäne der SAP-Variantenkonfiguration erläutert und die Paradigmen von Multi-Agenten-Systemen nach aktuellem Forschungsstand analysiert. In **Kapitel 3** erfolgt die Auswertung der Experteninterviews zur Festlegung der manuellen Baseline und der funktionalen Anforderungen. **Kapitel 4** widmet sich dem praktischen Selbstversuch, in dem die Auswahl des Frameworks, die Rollenspezifikation und die Integration in die SAP BTP dokumentiert werden. **Kapitel 5** umfasst die Evaluation des entwickelten Prototyps anhand der Forschungsfragen, um die messbaren Mehrwerte und Limitierungen gegenüber der Baseline darzustellen. Die Arbeit schließt in **Kapitel 6** mit einem Resümee und einem Ausblick auf zukünftige Forschungspotenziale.

Kapitel 2

Theoretische Grundlagen

Um die Konzeption und Entwicklung eines KI-gestützten Migrationssystems fundiert nachvollziehen zu können, ist ein tiefgreifendes Verständnis der zugrundeliegenden technologischen Domänen unerlässlich. Dieses Kapitel legt das theoretische Fundament der vorliegenden Arbeit, indem es die wesentlichen Konzepte der SAP-Variantenkonfiguration, der generativen künstlichen Intelligenz sowie der Orchestrierung von Multi-Agenten-Systemen systematisch aufarbeitet.

2.1 Domänenspezifische Grundlagen: SAP Variantenkonfiguration

2.1.1 Einführung in die logische Architektur der Variantenkonfiguration

In der modernen Fertigungsindustrie erfordert die Abbildung kundenindividueller Produkte hochflexible Systemstrukturen. Die grundlegende informationstechnische Idee der Variantenkonfiguration besteht darin, ein konfigurierbares Produkt – wie beispielsweise einen Personenkraftwagen – nicht als unzählige, separat gepflegte Endprodukte anzulegen, sondern als ein zentrales Grundmodell, das durch diverse Ausprägungen (wie Lackierung, Motorisierung oder Bereifung) spezifiziert wird. Diese Ausprägungen sind durch ein logisches Regelwerk miteinander verknüpft, um technische und kaufmännische Restriktionen abzubilden. Da die Kombination aller möglichen Merkmalsausprägungen die Anzahl der finalen Varianten schnell in die Millionen oder, in extremen Fällen der Anlagenfertigung, bis in den Bereich von Quadrillionen (10^{24}) treiben kann, ist die Anlage diskreter Stammdaten für jede einzelne physische Ausprägung in der IT-Praxis unmöglich (Blumöhr et al., 2023, S.

86).

Als systemtechnische Antwort auf diese kombinatorische Explosion fungiert im SAP-System der sogenannte *konfigurierbare Materialstamm* (KMAT). Dieser dient als universaler Platzhalter, unter dem die gesamte Variantenvielfalt eines Produkts informationstechnisch zusammengefasst wird (Blumöhr et al., 2023, S. 90). Um diese Vielfalt in einer sogenannten Bewertungsoberfläche abzubilden, greift die SAP-Architektur auf drei zentrale Strukturkomponenten zurück (Blumöhr et al., 2023, S. 90 - 96):

- **Merkmale:** Sie definieren die spezifischen Eigenschaften eines Produkts (wie beispielsweise die Lackierung oder die Traglast eines Gabelstaplers).
- **Variantenklassen:** Diese dienen der logischen Bündelung der Merkmale und werden direkt dem KMAT zugeordnet.
- **Konfigurationsprofil:** Dieses fungiert als übergeordnetes Steuerungselement, das die Stammdaten zusammenhält und den Konfigurationsprozess maßgeblich regelt.

Ein zentrales Paradigma der SAP-Variantenkonfiguration zur Bewältigung dieser Komplexität ist das sogenannte Maximal-Prinzip. Anstatt ein Produkt aus einzelnen Modulen additiv zu einem 100 %-Modell zusammenzusetzen, basiert die Architektur auf einem subtraktiven 150 %-Modell. Hierfür wird eine *konfigurierbare Maximalstückliste* (Super BOM) eingesetzt. Diese umfasst sämtliche Bauteile und Komponenten, die für jedwede theoretisch mögliche Variante des Produkts jemals benötigt werden könnten (Blumöhr et al., 2023, S. 169). Während des Konfigurationsprozesses können dieser Stückliste keine neuen Elemente hinzugefügt, sondern lediglich nicht benötigte Komponenten herausgefiltert werden.

Die logische Brücke zwischen der kaufmännischen Konfiguration und der technischen Fertigung bildet das *Beziehungswissen*. Metaphorisch gesprochen übersetzt dieses Regelwerk das kundenseitige „Wünschen“ in der Bewertungsoberfläche in das fertigungsseitige „Erhalten“ in Form der ausgedünnten Stückliste (Blumöhr et al., 2023, S. 101). Konkret wird dies durch Steuerungsmechanismen wie *Auswahlbedingungen* realisiert. Wenn ein Kunde beispielsweise aus einem Angebot von sechs Motoren einen spezifischen Antrieb wählt, determinieren die Auswahlbedingungen des Beziehungswissens, dass die fünf nicht gewählten Motoren aus der finalen Fertigungsstückliste eliminiert werden.

Die Zusammenhänge dieser fundamentalen Architekturkomponenten – vom konfigurierbaren Materialstamm über die Bewertungsoberfläche bis hin zur

Maximalstückliste und dem steuernden Beziehungswissen – sind in Abbildung ?? schematisch zusammengefasst.

(Hier kommt die Grafik rein)

2.1.2 Die klassische Variantenkonfiguration (LO-VC) im Kontext der Systemtransformation

Um die technologische Entwicklung der Variantenkonfiguration zu verstehen, ist zunächst eine Einordnung in die Evolution der SAP-Systemlandschaften erforderlich. Während das bisherige Kernprodukt *SAP ERP* (oft auch als *SAP R/3* bezeichnet) über Jahrzehnte den Standard darstellte, markiert *SAP S/4HANA* eine fundamentale Neuausrichtung der Softwarearchitektur, insbesondere durch die Nutzung der In-Memory-Datenbanktechnologie (Blumöhr et al., 2023, S. 28). In diesem Zuge wurde für die Variantenkonfiguration ein neues technologisches Fundament geschaffen.

Der Begriff LO-VC dient heute primär als Retronym, um die klassische Art der Konfiguration von der neuen AVC abzugrenzen (Blumöhr et al., 2023, S. 32). Obwohl LO-VC in aktuellen S/4HANA-Installationen (On-Premise und Private Cloud) aus Gründen der Abwärtskompatibilität weiterhin verfügbar ist, gilt sie technologisch als veralteter Standard.

LO-VC basiert dabei auf einem prozeduralen Paradigma, das historisch bedingt oft durch einen „Learning-by-Doing“-Ansatz in den Unternehmen gewachsen ist. Dies führte dazu, dass Modelle häufig unnötig komplex durch den Einsatz von Prozeduren und Aktionen anstatt leistungsfähigerer Constraints gestaltet wurden (Blumöhr et al., 2023, S. 58). Der Wechsel zu AVC wird in der Literatur daher nicht nur als technisches Upgrade, sondern als strategische Notwendigkeit beschrieben, die durch folgende Kernaspekte begründet wird:

- **Rückkehr zum Standard und Vereinfachung:** AVC stellt den künftigen Standard dar. Die Migration bietet Unternehmen die Chance, historisch gewachsene, schwer wartbare „Spaghetti-Logiken“ zu dekonstruieren und durch eine moderne, leichter wiederverwendbare Modellierung zu ersetzen (Blumöhr et al., 2023, S. 57).
- **Abbildung moderner Geschäftsmodelle:** Klassische LO-VC Strukturen sind primär auf den reinen Verkauf von Produkten (SD-Kontext) ausgelegt. Moderne Strategien wie das *Solution Bundling* (Kombination aus Produkt, Dienstleistung und Abonnements) lassen sich im Standard nur über AVC und die damit verknüpften neuen Objektträger wie die *Solution Quote* abbilden (Blumöhr et al., 2023, S. 59).

- **Technologische Überlegenheit:** AVC nutzt mit *Gecode* eine moderne Constraint-Solving-Engine, die im Gegensatz zur sequenziellen LO-VC-Logik analytische Prinzipien verfolgt und damit eine deutlich höhere Performance und Flexibilität bietet (Blumöhr et al., 2023, S. 59).
- **Zukunftsfähigkeit:** Innovative Technologien aus den Bereichen KI oder des *Machine Learning* (ML) sowie moderne Auswertungsmethoden via *CDS-Views* werden nativ für AVC entwickelt. Eine nachträgliche Integration in die veraltete LO-VC-Architektur gleicht lediglich einer „Renovierung“, die nie den Leistungsumfang einer Neuentwicklung erreichen kann (Blumöhr et al., 2023, S. 61).
- **Human-Resource-Aspekt:** Angesichts des Fachkräftemangels ist die Arbeit mit moderner Softwarearchitektur ein entscheidender Faktor bei der Gewinnung von Talenten. Die Wartung einer über 25 Jahre alten Softwarelösung ist im Vergleich zur Vorreiterrolle in einer intelligenten Fertigungsumgebung auf Basis aktueller Technologie weniger attraktiv für qualifizierten Nachwuchs (Blumöhr et al., 2023, S. 60).

Zusammenfassend lässt sich festhalten, dass der Verbleib in der LO-VC-Welt zwar kurzfristig Transformationsaufwände vermeidet, langfristig jedoch das Risiko erhöht, den Anschluss an technologische Innovationen zu verlieren und den Aufwand für eine spätere Migration durch das wachsende technologische Delta exponentiell zu vergrößern (Blumöhr et al., 2023, S. 61).

2.1.3 Die moderne Architektur (AVC) und das Constraint Paradigma

Die Antwort auf die konzeptionellen Schwächen und technischen Limitierungen der klassischen Variantenkonfiguration bildet die Einführung der AVC in SAP S/4HANA. Um den stetig wachsenden Anforderungen an Performance und Modellierungskomplexität (wie beispielsweise in den *Make-to-Order*- oder *Engineer-to-Order*-Szenarien) gerecht zu werden, vollzieht SAP mit AVC nicht nur ein syntaktisches Update, sondern einen fundamentalen informationstechnischen Paradigmenwechsel unter der Haube der Konfigurations-Engine.

Während das bisherige LO-VC-System, wie in Kapitel 2.1.2 beschrieben, auf einer iterativen, imperativen Abarbeitung von Befehlen und Prozeduren basierte, nutzt AVC einen grundlegend deklarativen Lösungsansatz. Das technologische Herzstück der AVC-Architektur bildet ein neu integrierter *Constraint Solver* namens *Gecode*, eine ursprünglich quelloffene Softwarebibliothek, die in Zusammenarbeit mit dem Fraunhofer-Institut ITWM tief

in den SAP-Standard eingebettet und für komplexe Produktkonfigurationen erweitert wurde (Blumöhr et al., 2023, S. 35).

Der Einsatz dieser Engine überführt den Konfigurationsprozess mathematisch in ein sogenanntes *Constraint Satisfaction Problem* (CSP). Ein CSP wird in der Informatik grundlegend definiert als ein System, das aus „einer Menge von Variablen, einer Menge von Domänen [sowie] einer Menge von Constraints, die zulässige Kombinationen von Werten spezifizieren“ (eigene Übersetzung nach Russell, 2010, S. 180) besteht.

Dieser deklarative Ansatz bringt tiefgreifende Auswirkungen auf die Benutzerführung und die Systemstabilität mit sich. Sobald ein Anwender während der Konfiguration in der Benutzeroberfläche einen Wert für ein bestimmtes Merkmal wählt, schränkt die Gecode-Engine die Domänen (Wertebereiche) aller anderen abhängigen Merkmale in Echtzeit ein. Anstatt einen Fehler erst nach der sequenziellen Abarbeitung einer Prozedur auszuwerfen, eliminiert der Solver unmögliche Auswahlmöglichkeiten präventiv aus dem Suchraum. Dies verhindert effektiv die Entstehung inkonsistenter Konfigurationen (Blumöhr et al., 2023, S. 35).

Darüber hinaus bietet AVC erweiterte Simulations- und Analysemöglichkeiten, wie beispielsweise den *Dependency Trace*. Dieser ermöglicht es Konstrukteuren und Modellierern, in einem *Inspector Panel* genau nachzuvollziehen, welches Constraint für die Einschränkung eines bestimmten Merkmals verantwortlich ist (Blumöhr et al., 2023, S. 37).

Zusammenfassend markiert AVC durch die Gecode-Engine den Wechsel von der Frage „Wie berechne ich die Variante Schritt für Schritt?“ hin zu der Frage „Welche globalen Bedingungen müssen für ein gültiges Endprodukt erfüllt sein?“. Genau dieser Paradigmenwechsel vom Imperativen zum Deklarativen stellt Unternehmen bei der Systemmigration vor die immense Herausforderung, historisch gewachsene, prozedurale Altlasten semantisch zu dekonstruieren und in das neue, netzwerkbasierte Constraint-Paradigma zu transformieren.

2.2 Generative künstliche Intelligenz im Software Engineering

2.2.1 Large Language Models für Quellcode

In den vergangenen Jahren haben KI-Systeme, insbesondere aus dem Bereich der generativen Künstlichen Intelligenz, einen rasanten und tiefgreifenden technologischen Aufstieg erlebt. Im Zentrum dieser Entwicklung stehen sogenannte *Large Language Models* (LLMs). Fundamental betrachtet handelt es

sich dabei um tiefe künstliche neuronale Netze, die auf massiven Textkorpora trainiert wurden, um das statistisch wahrscheinlichste nächste Zeichen (Token) in einer Sequenz vorherzusagen. Was als Werkzeug für rein statistische Textvervollständigungen begann, hat sich durch enorme Skalierung zu mächtigen kognitiven Architekturen entwickelt. Diese Modelle dringen zunehmend in den Bereich des Software Engineerings vor und verändern die Art und Weise, wie Software entworfen, analysiert und transformiert wird, grundlegend. Um das Potenzial dieser Technologie für automatisierte Migrationsprozesse zu verstehen, ist ein grundlegendes Verständnis der zugrundeliegenden mathematisch-technischen Prinzipien sowie der Methoden zu deren Steuerung erforderlich.

Die Transformer-Architektur und der Aufmerksamkeitsmechanismus

Die heutige Generation marktbeherrschender Sprachmodelle basiert fast ausnahmslos auf der wegweisenden Transformer-Architektur, die durch Vaswani et al. (2017) eingeführt wurde. Der entscheidende informationstechnische Durchbruch des Transformers liegt in der Abkehr von früheren Modellen wie rekurrenten neuronalen Netzen oder Faltungsnetzen. Während diese älteren Architekturen Eingabesequenzen – wie natürlichen Text oder Quellcode – streng sequenziell, mithin Zeichen für Zeichen oder Zeile für Zeile verarbeiten mussten, bricht der Transformer dieses lineare Paradigma auf.

Dies gelingt durch den sogenannten *Self-Attention-Mechanismus*. Dieser Mechanismus erlaubt es dem Modell, alle Elemente (Token) einer Sequenz zeitgleich und vollständig parallel zu verarbeiten. Mathematisch berechnet das Netzwerk für jedes Token innerhalb eines Kontextes einen Gewichtungsfaktor relativ zu allen anderen Token der Sequenz. Dadurch wird bestimmt, wie viel „Aufmerksamkeit“ einem Wort oder Code-Snippet im Verhältnis zum Rest des Textes beigemessen werden muss.

Im Kontext des Software Engineerings löst diese Architektur das fundamentale Problem weitreichender Abhängigkeiten (*Long-Range Dependencies*). Wenn beispielsweise eine Variable am Anfang einer umfangreichen Code-Datei deklariert und hunderte Zeilen später in einer komplexen logischen Bedingung aufgerufen wird, verkürzt der Transformer den informationellen Pfad im Netzwerk auf eine konstante Länge. Der Self-Attention-Mechanismus berechnet die direkte strukturelle Verknüpfung zwischen Deklaration und Aufruf unabhängig von ihrer physischen Distanz im Dokument. Quellcode wird somit nicht als linearer Text, sondern als ein Netz simultan interagierender logischer Einheiten interpretiert.

In-Context Learning und Few-Shot Prompting

Die Fähigkeit, tiefe semantische Strukturen zu erfassen, wirft die Frage auf, wie diese Modelle ohne immensen Rechenaufwand für spezifische, proprietäre Aufgabensteuerungen eingesetzt werden können. Historisch erforderte die Anpassung eines neuronalen Netzes an eine neue Domäne ein ressourcenintensives *Fine-Tuning*, bei dem die internen Gewichte des Modells durch überwachtes Lernen angepasst wurden. Moderne LLMs weisen jedoch ab einer kritischen Skalierungsgröße sogenannte emergente Fähigkeiten auf, die dieses Paradigma verschieben.

Wie durch Brown et al. (2020) nachgewiesen wurde, verhalten sich ausreichend große Sprachmodelle weitgehend aufgabenunabhängig (*task-agnostisch*). Anstatt die Modellparameter dauerhaft zu modifizieren, findet das Lernen direkt zur Laufzeit innerhalb des Eingabefensters statt – ein Konzept, das als *In-Context Learning* (ICL) bezeichnet wird. Das Modell nutzt sein im Pre-Training erworbenes, generalistisches Strukturwissen, um eine neue Aufgabe fließend und unmittelbar aus den im Prompt gegebenen Mustern abzuleiten.

Eine der mächtigsten Methoden des ICL ist das *Few-Shot Prompting*. Hierbei werden dem Modell im Eingabetext neben der eigentlichen Instruktion einige wenige konkrete Demonstrationen (Musterbeispiele) übergeben, die eine Eingabe und die dazugehörige korrekte Zielausgabe abbilden. Ohne dass ein einziges Gradientenupdate im neuronalen Netz stattfindet, adaptiert das LLM die logische Analogie zwischen den Beispielen und wendet diese auf die neue, ungesehene Zielaufgabe an. Für komplexe Domänen bietet dies den Vorteil, dass Modelle flexibel und agil gesteuert werden können, solange der Prompt die notwendigen logischen Leitplanken bereitstellt.

Das Paradoxon der Code-Übersetzung

Für den konkreten Einsatz generativer KI im Software Engineering liefert die umfassende Systematisierung von Zheng et al. (2023) eine wesentliche differenzierende Erkenntnis bezüglich der Modellauswahl. Entgegen der intuitiven Annahme, dass für Programmieraufgaben spezialisierte Modelle stets vorzuziehen sind, zeigt sich in empirischen Studien ein zweigeteiltes Bild.

Auf der einen Seite existieren hochspezialisierte, verhältnismäßig kleine Sprachmodelle (*Code LLMs*), die exklusiv auf riesigen Quellcode-Korpora trainiert wurden. Beim Agieren auf einer „grünen Wiese“ – mithin bei der isolierten Generierung von Standardfunktionen oder der automatisierten Erstellung von Unit-Tests – übertreffen diese dedizierten Code-Modelle die gigantischen, allgemeinsprachlichen Modelle oft deutlich. Sie weisen eine hohe syntaktische Präzision auf und sind in ihrer Ausführung hochgradig effizient.

Auf der anderen Seite verkehrt sich dieses Leistungsverhältnis ins Gegenteil, sobald die Aufgabe in den Bereich der Code-Übersetzung (*Code Translation*) fällt. Bei der Migration bestehender Logik von einer Quellsprache oder einem Paradigma in eine andere Zielarchitektur dominieren führende, generalistische Großmodelle (wie beispielsweise GPT-4) die spezialisierten Code-Modelle signifikant.

Die wissenschaftliche Begründung hierfür lässt sich aus der Natur der Problemstellung ableiten: Eine Software-Migration ist kein rein syntaktisches Substitutionsproblem, bei dem lediglich Schlüsselwörter ausgetauscht werden. Es handelt sich vielmehr um ein komplexes Abstraktions- und Logikproblem. Das Modell muss den semantischen Kern, die implizite Absicht und die vernetzten Bedingungen des Altsystems vollständig durchdringen, diese in eine sprachunabhängige logische Repräsentation abstrahieren und anschließend unter den restriktiven Bedingungen des Zielparadigmas neu formulieren. Für derart vielschichtige logische Schlussfolgerungen (*Reasoning*) ist das breite Weltwissen und die schiere kognitive Kapazität der großen, generalistischen Modelle unabdingbar, während rein syntaxfokussierte Code-Modelle an den abstrakten Paradigmenwechseln scheitern.

2.2.2 Limitationen monolithischer Ansätze

Obgleich die im vorherigen Abschnitt beschriebenen Fähigkeiten moderner Großmodelle zur semantischen Erfassung und Übersetzung von Quellcode beachtlich sind, stößt der praktische Einsatz in industriellen Software-Engineering-Projekten an fundamentale Grenzen. In der Praxis und der aktuellen Forschung werden Ansätze, bei denen eine komplexe Aufgabe in einer einzigen Interaktionsschleife – mithin über einen einzelnen, isolierten Eingabeaufforderungs-Befehl (*Single-Prompt* oder *One-Shot-Verfahren*) – gelöst werden soll, als *monolithische Ansätze* bezeichnet. Um die Notwendigkeit fortgeschrittener, verteilter Architekturen zu begründen, müssen die systemimmanenten Schwachstellen dieser monolithischen Herangehensweise auf zwei Ebenen analysiert werden: der algorithmischen Komplexität der Logikketten und der strukturellen Komplexität systemweiter Abhängigkeiten.

Die Komplexitätsfalle und das probabilistische Paradigma

Die primäre algorithmische Limitierung monolithischer Prompts bei logisch anspruchsvollen Aufgaben wurde bereits frühzeitig durch OpenAI im Rahmen der Vorstellung des Codex-Modells empirisch dokumentiert. Wie Chen et al. (2021) nachweisen, sind LLMs hervorragend in der Lage, isolierte, in sich

geschlossene Programmieraufgaben zu lösen, sofern die Spezifikation klar umrissen ist.

Das Leistungsvermögen bricht jedoch mathematisch nachweisbar ein, sobald eine Spezifikation lange Ketten sequenzieller oder verschachtelter Operationen verlangt (*Chaining-Problem*). Die Autoren zeigen, dass die Erfolgsquote des Modells mit zunehmender Anzahl logisch verknüpfter Bausteine innerhalb der Anforderung exponentiell abfällt (Chen et al., 2021). Für die Migration historisch gewachsener SAP-Regelwerke (wie dem klassischen *Variant Configuration*-Modell LOVC) ist diese Erkenntnis von kritischer Bedeutung: Solche Altsysteme bestehen typischerweise aus hochgradig verschachtelten, prozeduralen Abhängigkeiten und logischen Verzweigungen. Ein monolithischer Prompt zwingt das Modell, diese gesamte kognitive Last simultan zu verarbeiten und den Zielcode in einem einzigen autoregressiven Durchlauf fehlerfrei zu generieren. Dies überschreitet die inhärenten Fähigkeiten zur logischen Vorausschau des Modells, da es Token für Token auf Basis von Wahrscheinlichkeiten generiert, ohne eine echte logische Tiefenprüfung vorzunehmen.

Darüber hinaus beleuchtet das Paper eine fundamentale Schwäche des probabilistischen Charakters von Sprachmodellen. Die in diesem Paper vorgestellte Metrik *pass@1* – welche die Wahrscheinlichkeit beschreibt, dass ein einzelner generierter Code-Entwurf funktional korrekt ist – ist bei komplexen Aufgabenstellungen extrem gering. Erst durch das wiederholte Generieren unabhängiger Stichproben (*pass@k*) und eine anschließende externe Verifizierung steigt die Erfolgsquote signifikant an (Chen et al., 2021). Ein monolithischer Ansatz bietet jedoch keinerlei inhärente Mechanismen für ein solches iteratives *Sampling* oder eine automatisierte Qualitätskontrolle, wodurch fehlerhafte Syntax oder logische Fehlschlüsse unkorrigiert in das Zielergebnis einfließen.

Das Scheitern an systemweiten Abhängigkeiten und fehlendem Umgebungs-Feedback

Während die Arbeit von Chen et al. (2021) die Schwächen auf Ebene einzelner Funktionen aufzeigt, skaliert der aktuellere *SWE-bench*-Benchmark von Jimenez et al. (2024) diese Analyse auf das Niveau realer, industrieller Software-Repositories. Im Rahmen dieses Frameworks wurden führende Sprachmodelle mit echten, komplexen Problemstellungen aus großen Open-Source-Projekten konfrontiert, bei denen Änderungen nicht isoliert, sondern über Systemgrenzen hinweg durchgeführt werden mussten.

Die Ergebnisse dieses Benchmarks offenbaren ein klares Defizit monolithischer Großmodelle: Selbst hochgradig leistungsfähige, kommerzielle State-of-the-Art-Modelle wie GPT-4 oder die Claude Modellfamilie (Anthropic) waren im monolithischen *Single-Prompt*-Setting überfordert und konnten weit unter

5% der realen Software-Probleme erfolgreich lösen (Jimenez et al., 2024). Die Autoren identifizieren zwei Hauptursachen für dieses Scheitern:

1. **Mangelndes Verständnis globaler Abhängigkeiten:** Monolithische LLMs scheitern an der Identifikation von *Cross-File-Dependencies*. Sie sind zwar in der Lage, den lokal übergebenen Code-Kontext zu modifizieren, überblicken jedoch nicht, welche Auswirkungen diese Modifikation auf andere, im Kontext nicht enthaltene Klassen, Schnittstellen oder Datenstrukturen des Gesamtsystems hat. Bei einer SAP AVC-Migration, die auf einem global vernetzten, deklarativen *Constraint*-Netzwerk basiert, führt dieser blinde Fleck unweigerlich zu fatalen Inkonsistenzen im Gesamtsystem.
2. **Fehlen einer interaktiven Ausführungsumgebung:** Ein monolithischer Prompt agiert als reine Text-zu-Text-Transformation in einer isolierten Umgebung (Black-Box). Dem Modell fehlt jeglicher Zugriff auf einen Compiler, einen Interpreter oder Testprotokolle. In der realen Softwareentwicklung ist die fehlerfreie Code-Erstellung jedoch ein hochgradig interaktiver, iterativer Prozess. Ohne die Möglichkeit, den generierten Code testweise auszuführen und das daraus resultierende Compiler-Feedback dynamisch zu interpretieren, ist das Modell gezwungen, die syntaktische Korrektheit blind zu erraten.

Zusammenfassend lässt sich konstatieren, dass monolithische KI-Ansätze aufgrund der exponentiellen Fehleranfälligkeit bei langen Logikketten, des Mangels an systemweiter Kontextverarbeitung und der Isolation von echten Ausführungsumgebungen für komplexe Architektur-Transformationen ungeeignet sind. Diese wissenschaftlich dokumentierte Forschungslücke erfordert den Übergang von isolierten Prompts hin zu dynamischen, interaktiven Multi-Agenten-Systemen, die in der Lage sind, Aufgaben modular aufzuteilen und durch kontinuierliche Feedbackschleifen zu validieren.

2.3 Multi-Agenten-Systeme(MAS)

2.3.1 Definition und kognitive Architekturen

Um die im vorherigen Abschnitt dokumentierten Limitationen monolithischer Sprachmodelle – insbesondere den exponentiellen Leistungsabfall bei langen Logikketten und das Fehlen systemweiter Kontrollmechanismen – informationstechnisch aufzulösen, etabliert sich in der modernen Informatik das Paradigma der MAS. Die fundamentale Kernidee dieses Ansatzes besteht

darin, eine komplexe Gesamtaufgabe nicht einem einzelnen, allumfassenden Prompt zu übergeben, sondern die kognitive Last auf ein Kollektiv spezialisierter, künstlicher Entitäten – sogenannte KI-Agenten – zu verteilen, welche autonom und kollaborativ an der Erreichung eines vordefinierten Endziels arbeiten.

Der Begriff des LLM-basierten Agenten

In der klassischen Informatik wird ein Agent traditionell als eine Entität definiert, die in der Lage ist, ihre Umwelt durch Sensoren wahrzunehmen und auf diese mittels Aktuatoren einzuwirken, um rationale Entscheidungen zu treffen (Russell, 2010, S. 36). Im Kontext moderner generativer KI verschiebt sich dieses Konzept grundlegend: Das primäre „Gehirn“ und die zentrale Rechenmaschine des Agenten ist ein LLM.

Ein LLM-basierter Agent unterscheidet sich von einem klassischen, rein reaktiven Chatbot durch ein erhöhtes Maß an Autonomie, Intentionalität und Werkzeugnutzung. Technisch wird ein solcher Agent instanziiert, indem ein Basis-Sprachmodell mit einem spezifischen System-Prompt (*Instruction Engineering*) versehen wird, welcher seine Identität, seine Verhaltensregeln und seine Zielsetzung exakt formuliert. Darüber hinaus wird der Agent mit Schnittstellen zu externen Werkzeugen (*Tools*) oder Datenquellen ausgestattet.

Ein wesentliches Merkmal von Multi-Agenten-Strukturen ist die architektonische Flexibilität ihrer Orchestrierung. Es existiert kein starrer, mathematisch vorgegebener Standard, wie ein solches System strukturiert sein muss. Die Ausgestaltung des Kontrollflusses und der Kommunikationspfade obliegt dem Systemarchitekten. Dabei lassen sich zwei fundamentale Organisationsformen unterscheiden:

1. **Statische Orchestrierung:** Der Entwickler definiert im Vorfeld unveränderliche Kommunikationsketten und deterministische Ablaufpläne (z. B. Agent A muss seine Ausgabe zwingend an Agent B übergeben).
2. **Dynamische Autonomie:** Das Multi-Agenten-System erhält die inhärente Fähigkeit, zur Laufzeit eigenständig zu entscheiden, ob, wann und welche Sub-Agenten oder Werkzeuge zur Lösung eines auftretenden Teilproblems aktiviert werden müssen.

Die kognitive Kern-Anatomie nach Wang et al.

Obgleich die konkrete softwaretechnische Implementierung je nach verwendetem Entwicklungs-Framework variieren kann, hat sich in der wissenschaftlichen Literatur eine universelle, standardisierte kognitive Architektur für autonome

Agenten herauskristallisiert. Wie Wang et al. (2024) im Rahmen einer umfassenden Systematisierung nachweisen, lässt sich die funktionale Anatomie eines leistungsfähigen LLM-basierten Agenten trennscharf in vier interagierende Kernmodule unterteilen:

- **Profiling-Modul (Rollen-Spezifität):** Dieses Modul legt fest, welche spezifische Persona, Expertise und Verhaltensrestriktionen der Agent adaptiert. Durch die Zuweisung einer dedizierten Rolle (z. B. „SAP-AVC-Architekt“) wird der Suchraum des Sprachmodells im Pre-Training-Wissen fokussiert, was die Relevanz der Ausgaben drastisch erhöht.
- **Memory-Modul (Gedächtnis-Strukturen):** Das Gedächtnis sichert die zeitliche Konsistenz des Agenten. Es unterteilt sich in ein Kurzzeitgedächtnis (*Short-term Memory*), welches den aktuellen Konversationskontext puffert, und ein Langzeitgedächtnis (*Long-term Memory*), das es dem Agenten erlaubt, historische Informationen, erfolgreich gelöste Aufgaben oder domänenspezifisches Wissen dauerhaft zu persistieren.
- **Planning-Modul (Aufgaben-Dekonstruktion):** Komplexe Fragestellungen überfordern die direkte autoregressive Textgenerierung. Das Planungs-Modul befähigt den Agenten, ein globales Endziel autonom in überschaubare, logisch aufeinander aufbauende Teilaufgaben (*Sub-tasks*) zu zerlegen und Prioritäten zu setzen.
- **Action-Modul (Werkzeug-Exekution):** Dieses Modul bildet die Brücke zur Außenwelt. Es übersetzt die textuellen Intentionen des Sprachmodells in konkrete API-Aufrufe, Datenbankabfragen oder Code-Ausführungen und führt damit die im Planungs-Schritt definierten Aufgaben real aus.

Das ReAct-Paradigma zur dynamischen Fehlerkorrektur

Innerhalb des für Migrationsprozesse besonders kritischen Planungs- und Aktionsmoduls droht bei rein prozeduralen Prompts die Gefahr einer unkontrollierten Fehlerfortpflanzung. Um eine adaptive und verlässliche Problemlösungskompetenz zu garantieren, bildet das von Yao et al. (2022) eingeführte *ReAct*-Paradigma (*Reasoning and Acting*) das theoretische Fundament moderner Agenten-Kognition.

Das ReAct-Muster bricht die isolierte, statische Natur klassischer Sprachmodelle auf, indem es eine kontinuierliche, interaktive Schleife aus drei sequenziellen Phasen erzwingt:

Thought (Gedanke) $\longrightarrow$ Action (Handlung) $\longrightarrow$ Observation (Beobachtung)

In der *Thought*-Phase nutzt der Agent das LLM, um den aktuellen Zustand verbal zu analysieren, logische Schlussfolgerungen zu ziehen und eine spezifische Handlungsabsicht zu formulieren. In der darauffolgenden *Action*-Phase wird diese Absicht durch den Aufruf eines externen Werkzeugs – beispielsweise eines Compilers – exekutiert. Die entscheidende Innovation liegt in der *Observation*-Phase: Das reale Feedback der Umgebung (z. B. eine syntaktische Fehlermeldung des Compilers) wird abgefangen und als neuer Text-Kontext in das Modell zurückgespeist.

Das Modell verarbeitet diese Beobachtung in der nächsten *Thought*-Phase, reflektiert den Fehler aktiv und passt seinen nachfolgenden Aktionsplan dynamisch an. Dieser zirkuläre Prozess des *Self-Refinements* verhindert, dass der Agent – wie ein monolithischer Prompt – blind fehlerhafte Code-Strukturen halluziniert oder falsche Annahmen unkorrigiert fortschreibt. Das ReAct-Paradigma ermöglicht somit eine faktenbasierte, empirisch überprüfbare und selbstreparierende Code-Generierung, die für die präzise Transformation restriktiver SAP AVC-Constraint-Netzwerke unerlässlich ist.

2.3.2 Kollaborative Agenten-Muster

Nachdem im vorangegangenen Abschnitt die kognitive Funktionsweise und Anatomie einzelner KI-Agenten definiert wurde, befasst sich dieser Abschnitt mit der Orchestrierung mehrerer Agenten. Das Ziel *kollaborativer Agenten-Muster* ist es, die in Kapitel 2.2.2 beschriebenen Limitationen monolithischer Sprachmodelle – wie den Kontextverlust bei langen Logikketten und das Fehlen systemweiter Validierung – durch arbeitsteilige Architekturen gezielt auszumerzen. In der aktuellen Forschung haben sich hierfür verschiedene methodische Paradigmen etabliert.

Rollenspezialisierung und Dehalluzination

Eine der effektivsten Methoden zur Steigerung der Codequalität ist die Zuweisung harter, voneinander abgegrenzter Expertenrollen (wie beispielsweise Programmierer, Architekt und Tester), die sich in ihrer Zusammenarbeit an bewährten Vorgehensmodellen wie dem Wasserfallprinzip orientieren. Wie Qian et al. (2024) in ihrem *ChatDev*-Framework demonstrieren, ermöglicht diese Arbeitsteilung eine strukturierte Problemlösung, die monolithischen Ansätzen deutlich überlegen ist. Ein zentrales Konzept zur Fehlervermeidung ist dabei die *Communicative Dehallucination* (kommunikative Dehalluzination):

Anstatt bei unklaren Anforderungen fehlende Parameter oder Logiken blind zu erfinden (zu halluzinieren), werden Agenten so instruiert, dass sie proaktiv beim planenden Agenten nachfragen und detailliertere Spezifikationen einfordern, bevor Quellcode generiert wird.

Inception Prompting und System-Leitplanken

Um sicherzustellen, dass Agenten ihre zugewiesenen Rollen strikt beibehalten und zielgerichtet interagieren, bedarf es klarer Leitplanken. Li et al. (2023) führen hierfür im *CAMEL*-Framework das Konzept des *Inception Promptings* ein. Dabei werden Agenten durch asymmetrische System-Prompts zu Beginn der Konversation in eine feste Rolle gezwungen und mit strikten Verhaltensregeln ausgestattet. Um systemimmanente Gefahren wie das *Role Flipping* (den ungewollten Rollentausch zwischen Instrukteur und Assistent) oder sinnlose Kommunikations-Endlosschleifen zu verhindern, implementiert das System exakte Kommunikationsprotokolle und dedizierte Abbruch-Tokens (*Termination Tokens*).

Werkzeugnutzung und Code-Ausführung

Kollaboration in Multi-Agenten-Systemen beschränkt sich nicht ausschließlich auf den Austausch natürlicher Sprache, sondern umfasst auch die Interaktion mit externen Werkzeugen (*Tool Use*) und Laufzeitumgebungen. Das *AutoGen*-Framework von Wu et al. (2024) etabliert hierfür das Paradigma des *Conversation Programming*, bei dem der Kontrollfluss der Anwendung direkt durch den Agenten-Dialog gesteuert wird. Eine zentrale Rolle nehmen dabei sogenannte Proxy-Agenten ein. Deren einzige Aufgabe ist es, den von generierenden LLM-Agenten geschriebenen Quellcode auszuführen und im Falle von Syntax- oder Laufzeitfehlern die rohen Fehlermeldungen in den Chat zurückzuspiegeln. Dadurch entsteht eine unverzichtbare Brücke zwischen Textgenerierung und empirischer Validierung.

Iterative Fehlerkorrektur (Self-Refinement)

Die zielgerichtete Reaktion auf solche durch Proxy-Agenten aufgeworfenen Fehler erfordert strukturierte Kognitionsmechanismen. Shinn et al. (2023) beschreiben in ihrem *Reflexion*-Framework den Prozess der iterativen Fehlerkorrektur durch *Self-Refinement*. Meldet die Ausführungsumgebung einen Fehler, rät das Sprachmodell nicht blind auf eine neue Lösung. Stattdessen nutzt der Agent ein episodisches Gedächtnis, um den aufgetretenen Fehler verbal zu analysieren und zu reflektieren. Durch diesen „semantischen Gra-

dienten“ leitet der Agent eine konkrete Lösungsstrategie aus dem Fehler ab, speichert diese und passt den Code im nächsten iterativen Versuch präzise an.

Standard Operating Procedures und das Fließband-Paradigma

Die Synthese all dieser Muster führt zu hochkomplexen, skalierbaren Multi-Agenten-Systemen, die traditionelle Monolithen deklassieren. Den umfassenden empirischen Beleg hierfür liefert das *MetaGPT*-Framework von Hong et al. (2024). Die Architektur simuliert ein softwareentwickelndes Unternehmen als Fließband-System (*Assembly Line Paradigm*), das seine Agenten zur Einhaltung strenger menschlicher Standardarbeitsanweisungen (*Standard Operating Procedures, SOPs*) zwingt. Eine entscheidende Neuerung ist hier der strukturierte Dokumentenaustausch: Anstatt frei miteinander zu chatten, kommunizieren die Agenten über formalisierte Spezifikationen und Design-Dokumente. Die Studie beweist, dass diese strikt orchestrierte Architektur kaskadierende Halluzinationen stoppt und Monolithen in sämtlichen Benchmarks zur Code-Generierung signifikant schlägt.

2.3.3 Entwicklungs-Frameworks für MAS

Um die theoretischen Konzepte der kognitiven Anatomie (Kapitel 2.3.1) und der kollaborativen Muster (Kapitel 2.3.2) informationstechnisch umzusetzen, greift die moderne Softwareentwicklung auf spezialisierte Agenten-Frameworks zurück. Diese Bibliotheken abstrahieren die grundlegende Komplexität der Modell-Orchestrierung und stellen die softwaretechnische Infrastruktur (wie API-Anbindungen, Kontext-Fenster-Management und Logging) bereit.

Dass sich primär drei Frameworks etabliert haben, welche den wissenschaftlichen Diskurs sowie die industrielle Praxis maßgeblich prägen, lässt sich durch aktuelle quantitative Forschung belegen. In der ersten großangelegten empirischen Studie zur Evolution von Open-Source-Multi-Agenten-Systemen analysieren Liu et al. (2025) über 42.000 Commits führender Repositories. Die Autoren weisen anhand von GitHub-Metriken empirisch nach, dass insbesondere *LangChain* (als Basis-Ökosystem), *AutoGen* und *CrewAI* mit zehntausenden Entwickler-Interaktionen den Markt anführen und das rasanten Wachstum dieser Technologie seit 2023 treiben. Diese drei dominanten Plattformen unterscheiden sich in ihrer Architekturphilosophie wie folgt:

- **LangGraph:** Entwickelt als Erweiterung des populären *LangChain*-Ökosystems, modelliert dieses Framework Multi-Agenten-Systeme als zustandsbehaftete, zyklische Graphen. Jeder Agent oder jedes Werkzeug repräsentiert einen Knoten (*Node*), während die Kommunikationspfade

als Kanten (*Edges*) definiert werden (Chase, 2024). Es bietet maximale Flexibilität für komplexe, dynamische Kontrollflüsse, erfordert jedoch einen sehr hohen manuellen Programmieraufwand bei der Definition der Zustandsübergänge.

- **AutoGen:** Das von Microsoft entwickelte Framework stellt das Paradigma des „Conversation Programming“ in den Mittelpunkt. Agenten sind hier primär als dialogfähige Entitäten konzipiert, die Aufgaben lösen, indem sie direkt miteinander chatten. AutoGen zeichnet sich laut den Entwicklern durch die native Integration von *Proxy-Agenten* zur sicheren Code-Ausführung aus, bietet jedoch weniger strukturelle Leitplanken für streng prozedurale Abläufe (Wu et al., 2024).
- **CrewAI:** Dieses Framework wählt einen rollenbasierten Top-Down-Ansatz und ist architektonisch stark an das Fließband-Paradigma von MetaGPT angelehnt. Eine Entität (die *Crew*) orchestriert definierte Agenten, die als Mitglieder einer Hierarchie agieren und spezifische, sequentielle Aufgaben (*Tasks*) abarbeiten. Es fokussiert sich auf strikte Rollenverteilungen, Delegation und Standardarbeitsanweisungen (Liu et al., 2025).

Mapping von Theorie und Framework-Praxis

Die Entscheidung für ein solches Framework verschiebt die Aufgaben des Systemarchitekten. Die von Wang et al. (2024) definierte kognitive Kern-Anatomie ist in diesen Frameworks in der Regel als abstrahierte Code-Basis (*Out-of-the-Box*) verankert. So wird das Kurz- und Langzeitgedächtnis (*Memory*) oftmals automatisiert über integrierte Vektordatenbanken verwaltet. Auch das entscheidende *ReAct*-Paradigma (Yao et al., 2022) – mithin die Schleife aus Denken, Werkzeugnutzung und Beobachtung – läuft als nativer Hintergrundprozess ab, sobald einem Agenten ein Tool zugewiesen wird.

Obgleich diese Frameworks die informationstechnische Infrastruktur bereitstellen, abstrahieren sie lediglich universelle Prozesse und besitzen keinerlei domänenspezifisches Wissen über proprietäre SAP-Architekturen. Die maßgebliche Implementierungsleistung verlagert sich somit zwingend auf die manuelle Domänenspezialisierung. Unabhängig vom gewählten Framework verbleiben daher zwei Kernaufgaben in der Hand des Entwicklers:

1. **Systemdesign und Prompt-Engineering:** Der Entwickler muss evaluieren, welche der in Kapitel 2.3.2 beschriebenen theoretischen Konzepte am besten zur vorliegenden Problemstellung passen, und das Framework damit „befüllen“. Dies umfasst das harte Kodieren des

Profilings (Rollen und Identitäten), der Standardarbeitsanweisungen (SOPs) sowie der Delegationsregeln.

2. **Custom Tools (Werkzeug-Entwicklung):** Während das Framework die Logik des *Tool-Aufrufs* an sich abstrahiert, existieren für proprietäre Domänen – wie die SAP Variantenkonfiguration – keine vorgefertigten Werkzeuge. Die informationstechnische Brücke zwischen dem Agenten und der Zielumgebung (beispielsweise ein *Mock-Compiler* zur Syntax-Validierung von Constraint-Netzwerken) muss vollständig manuell entwickelt und als funktionale Schnittstelle in das Framework injiziert werden.

Mit den in diesem Kapitel erläuterten Konzepten – von den restriktiven Constraint-Netzwerken der SAP-Variantenkonfiguration über die Limitationen monolithischer Sprachmodelle bis hin zu den kollaborativen Architekturmustern moderner Agenten-Frameworks – sind nun alle wesentlichen technologischen Grundlagen für das tiefe Verständnis dieser Arbeit geschaffen. Im methodischen Gesamtkontext der Masterarbeit bildet diese fundierte Literaturanalyse die erste tragende Säule einer methodischen Triangulation. Die hier gewonnenen wissenschaftlichen Erkenntnisse dienen als theoretisches Alibi und Fundament, auf dem die nachfolgenden praktischen und architektonischen Entscheidungen aufbauen, um schließlich die eigens konzipierte Multi-Agenten-Migrationsarchitektur wissenschaftlich fundiert herzuleiten und zu begründen.

Kapitel 3

Related Work

Die Automatisierung komplexer Software-Migrationen erfordert ein tiefgreifendes Verständnis sowohl der proprietären Zieldomäne als auch der neuesten informationstechnischen Möglichkeiten. Da die wissenschaftliche Forschungslage an der direkten Schnittstelle zwischen proprietärer SAP-Migration und generativer Künstlicher Intelligenz bislang äußerst dünn besetzt ist, ordnet dieses Kapitel die vorliegende Arbeit anhand hochaktueller Publikationen aus angrenzenden Disziplinen in den Stand der Technik ein. Hierzu werden zunächst die spezifischen prozessualen Hürden bei der Transformation von SAP-Konfigurationsmodellen beleuchtet, um die Limitierungen aktueller Standardwerkzeuge aufzuzeigen. Daran anknüpfend wird der Einsatz von Large Language Models (LLMs) für die Code-Migration diskutiert, wobei insbesondere die empirisch belegten Grenzen monolithischer Ansätze bei strikten Architekturvorgaben herausgearbeitet werden. Abschließend wird der aktuelle State of the Art zu Multi-Agenten-Systemen (MAS) vorgestellt, um die methodische Forschungslücke für die semantische Migration von proprietärer Constraint-Logik logisch herzuleiten.

Die Migration von etablierten ERP-Systemen auf SAP S/4HANA zwingt Unternehmen zur Überführung ihrer historisch gewachsenen Produktmodelle von der klassischen Variantenkonfiguration (LO-VC) in die Advanced Variant Configuration (AVC). Matilainen (2024, S. 31) verdeutlicht in seiner Untersuchung zu S/4HANA-Produktdatenstrukturen, dass dieser Systemwechsel eine enorme Komplexität birgt, da die Anzahl der möglichen Konfigurationen mit der Menge der Merkmale und Optionen exponentiell ansteigt. Um diese Variantenvielfalt im neuen System technisch beherrschbar zu machen, ist eine präzise Restrukturierung der Datenmodelle unumgänglich.

Ein kritischer Erfolgsfaktor der Migration ist dabei die vorgelagerte Datenbereinigung. Wie Matilainen (2024, S. 52, S. 84) aufzeigt, müssen ungenutzte historische Datenelemente (Legacy Data) vor der Systemumstellung zwin-

gend identifiziert und entfernt werden, um Strukturen zu vereinfachen und Prozesse im Zielsystem nicht zu belasten. Obgleich SAP für diesen Übergang Standardwerkzeuge wie Transition Workspaces oder das Migration Cockpit zur Verfügung stellt, liegt deren Fokus primär auf der Massendatenübernahme und technischen Synchronisation. Wie jedoch das im Rahmen dieser Arbeit durchgeführte Experteninterview (siehe Kapitel 3) belegt, existiert bei der logischen Transformation eine signifikante Lücke: Die Standardwerkzeuge arbeiten weitestgehend deterministisch und sind nicht in der Lage, die notwendige semantische Interpretation von prozeduralem Beziehungswissen in eine rein deklarative AVC-Logik autonom zu leisten. Diese inhaltliche Restrukturierung sowie die Bereinigung kognitiver Altlasten im Code verbleiben somit als manueller, ressourcenintensiver Flaschenhals, für den die aktuelle industrielle Praxis keine automatisierten Lösungen bereithält.

Die rasanten Fortschritte im Bereich der Large Language Models (LLMs) haben die Möglichkeiten der automatisierten Code-Generierung und -Transformation grundlegend erweitert (Karabiyik, 2025). Dennoch belegen aktuelle Forschungsarbeiten, dass der direkte Einsatz von LLMs für die Migration komplexer Altsysteme – oft als „Single-Prompt-Ansatz“ oder „Monolithic Prompting“ bezeichnet – in realen Produktionsumgebungen kaum praktikable Ergebnisse liefert. Die Ursache hierfür liegt primär in der Unfähigkeit monolithischer Ansätze, spezifische Architekturvorgaben und interne Framework-Abhängigkeiten zu berücksichtigen. Moti et al. (2026) demonstrieren in ihrer Untersuchung zum Framework LegacyTranslate, dass naive LLM-Übersetzungen bei der Migration industrieller Codebasen eine Kompilierrate von 0 % erreichen können. Während der generierte Code zwar syntaktisch korrekt erscheint, scheitert er an der fehlenden Integration notwendiger interner Schnittstellen (APIs) und domänenspezifischer Logik (Moti et al., 2026). Ähnliche Defizite dokumentieren Xu et al. (2025) im Kontext von MANTRA: Während spezialisierte Systeme hohe Erfolgsquoten erzielen, erreichen monolithische Basismodelle bei komplexen strukturellen Änderungen lediglich eine Erfolgsquote von 8,7 %.

Darüber hinaus identifizieren Karabiyik (2025) und Oueslati et al. (2026) konzeptionelle Schwächen in der Kontrollierbarkeit solcher „One-Shot“-Ansätze. Ohne eine Aufteilung in modulare Arbeitsschritte bleibt der Transformationsprozess eine intransparente „Blackbox“, die keine funktionalen Garantien bietet und bei längeren Interaktionen zu Halluzinationen neigt (Karabiyik, 2025; Oueslati et al., 2026). Um diese Unzulänglichkeiten zu überwinden, rückt die aktuelle Forschung zunehmend Multi-Agenten-Systeme (MAS) in den Fokus. Diese basieren auf dem Prinzip der verteilten Kognition, bei der der Transformationsprozess in logisch abgegrenzte Teilaufgaben zerlegt wird, die von kooperierenden, spezialisierten Agenten autonom bearbeitet werden.

Dabei belegen aktuelle Arbeiten zunächst im Bereich des automatisierten Software-Refactorings – also der strukturellen Optimierung innerhalb derselben Programmiersprache –, die prozessuale Überlegenheit modularer Architekturen. Das Framework RefactorGPT überführt den Refactoring-Prozess durch eine sequentielle Orchestrierung in einen kontrollierbaren Workflow (Karabiyik, 2025). Die methodische Stärke dieser Rollenverteilung wird durch das Framework RefAgent bestätigt: Ein System aus spezialisierten Agenten für Planung, Ausführung und iterative Fehlerbehebung ist in der Lage, die Kompiliererfolgsrate im Vergleich zu Single-Agent-Ansätzen signifikant zu steigern und Halluzinationen zu reduzieren (Oueslati et al., 2026). Ein zentrales Merkmal dieser Prozess-Sicherheit ist die Integration autonomer Feedback-Schleifen. Das System MANTRA nutzt hierfür einen „Repair Agent“, der mittels „Verbal Reinforcement Learning“ Compiler-Fehler korrigiert, wodurch die Erfolgsquote bei komplexen Transformationen auf 82,8 % gesteigert werden konnte (Xu et al., 2025).

Während diese Refactoring-Ansätze die grundsätzliche Effizienz der verteilten Agenten-Kollaboration validieren, demonstriert das Projekt LegacyTranslate (Moti et al., 2026), dass sich diese Architektur auch auf die ungleich komplexere Herausforderung harter Cross-Language-Migrationen anwenden lässt. Im Gegensatz zur Umstrukturierung innerhalb einer Sprache erfordert die Migration von PL/SQL zu Java einen Paradigmenwechsel, der dem Übergang von LO-VC zu AVC ähnelt. LegacyTranslate verdeutlicht hierbei die Notwendigkeit des „API Grounding“, um den Zielcode aktiv an interne Architekturvorgaben anzupassen. Ähnlich wie dieses Grounding sicherstellt, dass interne Bibliotheken korrekt eingebunden werden, muss ein Agenten-System im SAP-Umfeld sicherstellen, dass die Transformation strikt den „Clean Core“-Richtlinien folgt – etwa durch die gezielte Vermeidung proprietärer Z-Funktionsbausteine zugunsten des SAP-Standards.

Trotz der empirisch nachgewiesenen Überlegenheit von MAS bei der Code-Übersetzung offenbart die aktuelle Literatur eine signifikante Lücke hinsichtlich proprietärer ERP-Ökosysteme. Während etablierte Frameworks wie MANTRA oder LegacyTranslate primär auf imperative oder objektorientierte Standardsprachen (wie Java oder PL/SQL) fokussiert sind, fehlt bislang die Anwendung kollaborativer KI-Architekturen auf die domänenspezifische Constraint-Logik der SAP. Es ist wissenschaftlich bisher nicht belegt, inwiefern Multi-Agenten-Systeme in der Lage sind, historisch gewachsene Regelwerke semantisch in ein deklaratives Constraint Satisfaction Problem (CSP) zu übersetzen. Bisherige MAS-Ansätze bieten keine architektonische Antwort auf dieses fachliche Umdenken, das LO-VC-Vorbedingungen in ein mathematisches Lösungsnetzwerk überführt.

Genau an dieser methodischen Schnittstelle setzt die vorliegende Arbeit an.

Sie schließt die identifizierte Forschungslücke, indem sie die Konzepte arbeitsteiliger Agenten-Architekturen auf die spezifischen Herausforderungen der SAP-Variantenkonfiguration überträgt. Durch die Konzeption und Evaluierung eines spezialisierten Multi-Agenten-Systems zur semantischen Migration von LO-VC nach AVC wird ein Lösungsansatz entwickelt, der den aktuellen manuellen Flaschenhals der Logik-Transformation systematisch adressiert und das Systemverhalten aktiv an die moderne, deklarative Zielarchitektur anpasst.

Kapitel 4

Durchführung und Auswertung des Experteninterviews

In diesem Kapitel wird die methodische Durchführung und Auswertung des qualitativen Experteninterviews dokumentiert. Die gewonnenen Erkenntnisse dienen innerhalb des gestaltungsorientierten Forschungsdesigns als empirische Brücke zum organisationalen und menschlichen Umfeld (*Environment*). Sie bilden das Fundament zur Festlegung der manuellen Baseline sowie zur Definition der funktionalen Anforderungen an das Multi-Agenten-System.

4.1 Zielsetzung und Erkenntnisinteresse

Die Relevanz sowie die informationstechnische Kritikalität der Migration von LO-VC zu AVC wurden bereits in Kapitel 2.1.2 dargelegt. Für die theoriegeleitete Konzeption eines MAS ist die bloße Analyse theoretischer Syntaxunterschiede jedoch unzureichend. Spezifische, historisch gewachsene Problemstellungen in der Entwicklungspraxis weisen oft eine hohe Kontextabhängigkeit auf und lassen sich nicht rein analytisch aus Systemdokumentationen ableiten. Zudem mangelt es in der Literatur an quantitativen Daten bezüglich des tatsächlichen manuellen Migrationsaufwands.

Folglich ist die Durchführung eines Experteninterviews zur Beantwortung der Unter-Forschungsfragen zwingend erforderlich. Das primäre Erkenntnisinteresse konzentriert sich auf zwei Dimensionen:

- **Quantifizierung der Baseline (FF1):** Erhebung valider zeitlicher Metriken und prozessualer Schritte der manuellen Migration, um einen empirischen Vergleichsmaßstab für die spätere Evaluation des Artefakts zu etablieren.

- **Anforderungsdefinition und Scoping (FF3):** Identifikation impliziter Praxisprobleme, kognitiver Hürden und typischer Fehlerquellen im msg-Kontext, um daraus spezialisierte agentische Rollen und funktionale Anforderungen für die Systemarchitektur abzuleiten.

4.2 Methodisches Forschungsdesign

Um dem Anspruch forschungsmethodischer Strenge (*Rigor*) gerecht zu werden, ist das Experteninterview als integraler Bestandteil der explorativen Fallstudie nach den Richtlinien von Runeson und Höst (2009) gestaltet. Aufgrund der Komplexität der Code-Transformation, die maßgeblich durch undokumentiertes Erfahrungswissen (*Tacit Knowledge*) geprägt ist, bedarf es zur fundierten Durchdringung der Problemstellung eines qualitativen Forschungsansatzes (Seaman, 1999).

Als Erhebungsinstrument wird das semi-strukturierte Interview gewählt. Dieses Format kombiniert offene und spezifische Fragen in einem flexiblen Leitfaden. Dadurch wird sichergestellt, dass einerseits antizipierte Daten – wie die zeitlichen Metriken für die Baseline – systematisch erfasst werden, während andererseits ausreichend Raum für unvorhergesehene qualitative Erkenntnisse aus der Migrationspraxis bleibt (Seaman, 1999). Während in sozialwissenschaftlichen Disziplinen oft ein geringes Vorwissen der interviewenden Person gefordert wird, postulieren Hove und Anda (2005) für den Bereich des Software Engineerings die Notwendigkeit eines tiefgreifenden technischen Vorverständnisses, um fachspezifische Nuancen während des Gesprächs korrekt zu erfassen und zu vertiefen.

Die Auswertung des Datenmaterials folgt einem systematischen Mixed-Methods-Ansatz, der Methoden der qualitativen und quantitativen Inhaltsanalyse kombiniert. Der Codierungsprozess gliedert sich in Anlehnung an Mayring (2014) in zwei Phasen:

1. **Deduktive Kategorienanwendung:** Vorab definierte Codes, die direkt aus den Forschungsfragen abgeleitet wurden (z. B. manuelle Prozessschritte, Zeitaufwand), werden theoriegeleitet auf das Textmaterial angewendet.
2. **Induktive Kategorienbildung:** Ergänzende Codes für unvorhergesehene praxisnahe Schmerzpunkte oder spezifische Syntaxprobleme werden iterativ direkt am empirischen Material entwickelt.

Im Sinne der in Kapitel 1.3 beschriebenen Datentriangulation bilden die Ergebnisse dieses Kapitels eine von drei empirischen Säulen. Sie werden im

weiteren Verlauf der Arbeit mit den Erkenntnissen der Archivdokumentenanalyse sowie den Ergebnissen des praktischen Selbstversuchs synthetisiert, um die initiale, valide Architekturversion 1 (V1) des MAS zu konstruieren. Um hierbei ein zentrales Qualitätsmerkmal qualitativer Fallstudien zu erfüllen, wird über den gesamten Auswertungsprozess – von der Audioaufzeichnung über das codierte Transkript bis hin zu den daraus resultierenden Field Memos und funktionalen Anforderungen – eine lückenlose Beweiskette (*Chain of Evidence*) aufrechterhalten (Runeson & Höst, 2009). Dies garantiert die transparente und wissenschaftlich nachvollziehbare Ableitung jeder späteren agentischen Rolle aus den empirischen Rohdaten.

4.3 Datenerhebung und Auswertungsprozess

Das Experteninterview wurde am 17.03.2026 mit einem erfahrenen Product Owner und Softwareentwickler der *msg systems ag* durchgeführt. Die Fachexpertise des Interviewpartners umfasst eine zwölfjährige Tätigkeit als Maschinenbauingenieur sowie eine mehr als sechsjährige Spezialisierung auf dem Gebiet der SAP-Variantenkonfiguration. Seit knapp vier Jahren liegt sein primärer Projektschwerpunkt auf der Betreuung von Kundenmigrationen von LO-VC nach AVC im msg-Kontext. Das Gespräch wies eine Dauer von rund 45 Minuten auf und wurde vom Autor dieser Arbeit auf Basis des vorab definierten Leitfadens moderiert.

Mit ausdrücklichem Einverständnis des Experten wurde das Gespräch digital aufgezeichnet und über die integrierte Funktionalität von Microsoft Teams automatisch transkribiert. Zur Sicherung der Datenqualität und Wahrung der wissenschaftlichen Integrität durchlief das Rohmaterial im Nachgang einen dreistufigen Aufbereitungsprozess:

1. **Anonymisierung:** Manuelle Entfernung aller personenbezogenen und projektspezifischen Identifikationsmerkmale zur Gewährleistung der Vertraulichkeit.
2. **KI-gestützte Glättung:** Bereinigung des Transkripts von rein sprachlichen Redundanzen (z. B. Füllwörter, Satzabbrüche) mittels generativer KI zur Optimierung der Lesbarkeit.
3. **Manuelle Verifikation:** Eine vollständige, zeilenweise Überprüfung des geglätteten Textes durch den Autor, um sicherzustellen, dass durch die Bearbeitung keinerlei inhaltliche oder technologische Verzerrungen eingeführt wurden.

Das finale, geglättete Transkript (siehe Anhang 6.2.4) bildete die Datengrundlage für das in Kapitel 4.2 beschriebene Codierungsverfahren. Die deduktive Basis-Codeliste strukturierte das Material zunächst in die Hauptkategorien *Prozessschritte der Migration*, *Zeitaufwand und quantitative Metriken* sowie *kognitive Hürden*. Durch den induktiven Codierdurchlauf konnten zusätzliche, spezifische Hürden – wie etwa das fehleranfällige manuelle Parsing komplexer SAP-Prozeduren – als kritische Systemanforderungen extrahiert werden. Die im Folgenden dargestellten Ergebnisse und Memos bilden die aggregierte Synthese dieses empirischen Prozesses.

4.4 Ergebnisse der Befragung

Die nachfolgende Darstellung fasst die zentralen Erkenntnisse der qualitativen Inhaltsanalyse zusammen, welche durch die Synthese der Field Memos in thematische Schwerpunkte gegliedert wurden. Diese deskriptiven Ergebnisse bilden den empirischen Ist-Zustand des Migrationsprozesses ab und dienen als Grundlage für die spätere Anforderungsanalyse und die Definition der menschlichen Baseline.

4.4.1 Kognitive Altlasten und historischer Code

Die Auswertung der Interviewdaten verdeutlicht, dass historisch gewachsene LOVC-Modelle durch eine hohe Dichte an ungenutzten Altlasten charakterisiert sind. Merkmale, Prozeduren und Beziehungswissen verbleiben oft im System, ohne im aktuellen Modellierungskontext eine funktional Relevanz zu besitzen. Zudem führen über Jahre stetig anwachsende Werteräume zu einer erhöhten Komplexität. Für den Migrationsprozess bedeutet dies, dass eine unreflektierte 1:1-Konvertierung in den Advanced Variant Configurator (AVC) aufgrund dieser behindernden Altlasten in der Praxis unzureichend ist. Für den Entwickler resultiert daraus eine erhebliche kognitive Belastung, da eine zeitintensive manuelle Analyse zur Trennung von aktiver Logik und „totem Code“ zwingend erforderlich ist, bevor die eigentliche syntaktische Transformation beginnen kann.

4.4.2 Paradigmenwechsel und syntaktisches Umdenken

Über die rein syntaktische Ebene hinaus zeigt die Befragung einen fundamentalen Paradigmenwechsel in der Modellierungstechnik auf. Dieser wird im industriellen Kontext unter anderem durch moderne SAP-Architekturvorgaben

wie die „Clean Core“-Strategie flankiert, wodurch proprietäre Eigenentwicklungen wie Z-Functions im AVC nicht mehr unterstützt werden. Da sich das Systemverhalten im AVC selbst bei identischer Syntax massiv ändern kann – etwa durch den Zwang zu klassifizierten Merkmalen oder die globale Wirkung von Constraints – führt eine bloße Übernahme alter Logiken häufig zu abweichenden Konfigurationsergebnissen oder Performance-Einbußen. Der Migrationsprozess erfordert daher ein aktives Umdenken: Entwickler müssen die ursprüngliche Intention des Legacy-Codes in Gänze verstehen und diese semantisch mit den neuen Bordmitteln des AVC abbilden.

4.4.3 Fragmentierung manueller Prozessschritte

Trotz der Existenz von Standard-Tools zur Massenanlage von Objekten bleibt der Migrationsprozess in weiten Teilen fragmentiert und durch manuelle Arbeitsschritte geprägt. Während Automatisierungen die Anlage von Basisdaten übernehmen können, verbleibt die komplexe intellektuelle Durchdringung der Logik beim Entwickler. Dieser muss Vorbedingungen manuell auswerten, in tabellenbasierte Strukturen überführen und abschließend auf Konsistenz prüfen. Dieser Workflow erzwingt häufig Medienbrüche und degradiert den Experten zu einer „manuellen Übersetzungsmaschine“, was die Fehleranfälligkeit erhöht und den Prozess verlangsamt.

4.4.4 Limitierungen bestehender Standard-Tools

Die Auswertung zeigt, dass bestehende Standardlösungen wie das „SAP Migration Cockpit“ primär die rein technischen Datenübernahmen (wie die Massenanlage von Objekten) abdecken. Bei der komplexen Logik-Transformation entstehen jedoch funktionale Lücken. Der Experte äußert den Bedarf nach einer prozessualen Unterstützung auf einer „beratenden Flugebene“, welche Zusammenhänge aufzeigt und Optimierungspotenziale identifiziert. Es mangelt im aktuellen Ist-Zustand an Werkzeugen zur automatisierten Kommentierung von historischem Code, zur tiefgehenden Analyse von Z-Functions sowie an proaktiven Ratschlägen zur Modellstrukturierung. Eine rein statische Code-Konvertierung erweist sich für den Gesamtprozess somit als unzureichend.

4.4.5 Manueller Aufwand und Fehleranfälligkeit im Testprozess

Ein signifikanter Flaschenhals wurde im Bereich der Qualitätssicherung identifiziert. Das Testen migrierter Modelle erfolgt gegenwärtig weitgehend manuell,

da bestehende Testtools oft nicht in der Lage sind, logische Veränderungen zwischen Alt- und Neusystem korrekt zu interpretieren. Dies führt zu einem massiven Aufwand beim händischen Abgleich von Testaufträgen und birgt ein entsprechendes Potenzial für menschliche Flüchtigkeitsfehler in der abschließenden Validierungsphase.

4.4.6 Zeitliche Metriken und Identifikation von Flaschenhälsen

Zur Definition einer menschlichen Baseline wurden im Rahmen der Befragung konkrete zeitliche Metriken erhoben. Die Migration eines mittleren Modells beansprucht demnach etwa drei bis vier Personentage, wobei Extremfälle bei komplexen Strukturen bis zu mehrere Monate beanspruchen können. Als primärer zeitlicher Flaschenhals wurde die Transformation der Vorbedingungswelt identifiziert, die etwa zwei Drittel (66 %) der gesamten Arbeitszeit beansprucht. Weitere 40 % des Aufwands entfallen auf die vorbereitende Analyse und den Aufbau unterstützender Strukturen wie Excel-Tabellen. Diese quantifizierten Werte bilden die Vergleichsvariablen für die spätere Evaluierung.

Die hier dargelegten empirischen Befunde skizzieren die aktuellen prozessualen und kognitiven Hürden der Migration. Sie bilden die konzeptionelle Ausgangslage, um im folgenden Unterkapitel konkrete Implikationen und Anforderungen an die zu entwickelnde Systemarchitektur abzuleiten.

4.5 Implikationen für die Systemarchitektur (Ideales Zielbild)

Basierend auf den empirischen Befunden der qualitativen Inhaltsanalyse lassen sich spezifische Anforderungen und konzeptionelle Leitplanken für die Gestaltung der Systemarchitektur ableiten. In diesem Kapitel werden ausschließlich jene Implikationen dargelegt, die direkt aus der Expertenbefragung hervorgehen. Sie repräsentieren den maximalen *Business Need* der industriellen Praxis und skizzieren das theoretische Idealbild des Systems.

4.5.1 Funktionale Anforderungen aus der Expertenperspektive

Aus den identifizierten prozessualen Hürden ergeben sich aus Sicht der Praxis folgende zentrale funktionale Wünsche an das Systemdesign:

- **Semantische Analyse und Code-Bereinigung:** Da historische Altlasten die Migration behindern, sollte die Architektur eine Komponente zur Identifikation von ungenutzten Merkmalen und Prozeduren enthalten. Ziel ist eine Reduktion der kognitiven Last durch eine automatisierte Vorab-Bereinigung des Legacy-Codes.
- **Automatisierte Logik-Transformation:** Angesichts des zeitlichen Flaschenhalses bei der Erstellung von Tabellen und Constraints muss der Fokus auf der automatisierten Generierung dieser Strukturen liegen. Das System sollte idealerweise in der Lage sein, die logische Restrukturierung von Vorbedingungen zu übernehmen, um den manuellen Aufwand drastisch zu minimieren.
- **Konsistenzprüfung im Testprozess:** Um die Fehleranfälligkeit bei der Validierung zu senken, wünscht sich die Praxis eine Funktion zum Mapping alter Testkonfigurationen auf die Zielmodelle, die logische Abweichungen zwischen den Systemwelten erkennt und den manuellen Testaufwand reduziert.

4.5.2 Architektonische Paradigmen und Beratungsansatz

Der identifizierte Paradigmenwechsel in der Modellierungstechnik bedingt zudem spezifische architektonische Eigenschaften:

- **Interaktive Assistenz (Consulting-Ansatz):** Das System sollte nicht als rein deterministischer Konverter agieren. Gefordert ist eine interaktive Ebene, die Design-Entscheidungen begründet und dem Nutzer beratend zur Seite steht, um den Kontext der Transformation transparent zu machen.
- **Einhaltung von AVC-Modellierungsrichtlinien:** Die Architektur sollte standardkonformen AVC-Zielcode erzeugen. Dabei gilt es, grundlegende Performance-Best-Practices der SAP – wie den Verzicht auf obsoletere Z-Funktionen zugunsten nativer Standard-Funktionen – direkt im Transformationsschritt zu berücksichtigen, um eine fehlerfreie Lauffähigkeit in der neuen Konfigurations-Engine sicherzustellen.

4.5.3 Hypothese der Multi-Agenten-Rollen

Die Heterogenität der Anforderungen – von der Analyse komplexer Z-Functions bis hin zur Validierung von Test-Sets – legt eine Verteilung der Aufgaben auf

spezialisierte Rollen nahe. Ein monolithischer Ansatz erscheint unzureichend, um die benötigte Tiefe in den Bereichen Analyse, Beratung, Synthese und Test gleichzeitig abzudecken. Daraus ergibt sich die konzeptionelle Hypothese einer Multi-Agenten-Architektur, in der idealerweise spezialisierte Agenten (wie z. B. Analyst, Syntaktiker, Tester und Consultant) kollaborativ zusammenarbeiten.

4.5.4 Quantitative Zielvorgaben für die Systemvalidierung

Die erhobenen Zeitmetriken definieren die harten Erfolgskriterien für die spätere Evaluierung der Architektur. Das System muss gegen die menschliche Baseline von drei bis vier Tagen pro Modell validiert werden. Insbesondere soll nachgewiesen werden, dass der primäre Flaschenhals der Vorbedingungs-Transformation (66 % des Gesamtaufwands) sowie die vorbereitende Analysephase (40 % des Aufwands) durch die automatisierte Unterstützung messbar reduziert werden können.

4.5.5 Überleitung zum methodischen Realitätsabgleich

Die hier dargelegten interviewbasierten Implikationen definieren das angestrebte Idealbild der Praxis. Im Sinne des methodischen *Design Science Research* Vorgehens (vgl. Kapitel 1.3) darf dieses vollumfängliche Anforderungsprofil jedoch nicht unreflektiert in den Systementwurf übernommen werden. Vielmehr bedarf es nun eines technologischen Realitätsabgleichs. Im darauffolgenden Kapitel (Selbstversuch) werden diese idealisierten Forderungen – insbesondere hochkomplexe Wünsche wie die automatisierte Test-Migration oder weitreichende Consulting-Funktionen – auf Code-Ebene auf ihre tatsächliche softwaretechnische Machbarkeit geprüft. Der Selbstversuch fungiert somit als methodischer Filter, um valide zu entscheiden, welche Anforderungen in die finale Agenten-Architektur (Version 1) aufgenommen werden und welche Aspekte aufgrund unverhältnismäßiger technologischer Komplexität bewusst aus dem *Scope* dieser Masterarbeit exkludiert werden müssen.

Kapitel 5

Durchführung und Auswertung des praktischen Selbstversuchs

Nachdem im vorangegangenen Kapitel durch das Experteninterview die praxisnahen Schmerzpunkte und die zeitliche Baseline für den manuellen Migrationsaufwand etabliert wurden, bildet die in diesem Kapitel beschriebene Machbarkeitsanalyse (Feasibility Check) kombiniert mit einer manuellen Code-Analyse den zweiten empirischen Baustein der Datentriangulation. Im Kontext des Design Science Research Frameworks nach Hevner et al. (2004) dient dieser Schritt dazu, das technologische Umfeld (Environment) tiefgreifend zu durchdringen. Während Interviews das subjektiv wahrgenommene, prozessuale Wissen der Fachexperten abbilden, fokussiert diese Phase die physische informationstechnische Ebene der rohen LO-VC-Datenstrukturen.

Das primäre Erkenntnisinteresse dieses Kapitels ist dabei zweidimensional: Zunächst erfolgt eine empirische Scope-Definition (Machbarkeitsanalyse), um die theoretischen Anforderungen aus der Praxis auf ihre technologische Automatisierbarkeit im Rahmen dieser Arbeit zu prüfen. Im direkten Anschluss erfolgt der praktische Selbstversuch zur Extraktion exakter syntaktischer Transformationsregeln, welche die kognitive Blaupause für das Multi-Agenten-System bilden.

5.1 Technologischer Realitätsabgleich und empirische Scope-Definition

Die im vorangegangenen Kapitel dargelegten interviewbasierten Implikationen (vgl. Kapitel 4.5) repräsentieren den maximalen *Business Need* aus Sicht der industriellen Praxis und skizzieren ein theoretisches Idealbild des zukünftigen Systems. Im Sinne des übergeordneten *Design Science Research* Paradigmas

nach Hevner et al. (2004) darf dieses vollumfängliche Anforderungsprofil jedoch nicht unreflektiert als feste Vorgabe für die Artefaktkonstruktion dienen. Vielmehr bedarf es vorab einer fundierten Machbarkeitsprüfung (*Feasibility Check*), um den exakten funktionalen und architektonischen Umfang (*Scope*) des zu entwickelnden Prototyps abzugrenzen.

Dieser Abschnitt fungiert als methodischer Filter zwischen den idealisierten Wünschen der Praxis und der technologischen Realität im Rahmen einer Masterarbeit. Die fünf Kernimplikationen aus Kapitel 4.5 werden nachfolgend systematisch evaluiert, um zu bestimmen, welche Aspekte im darauffolgenden praktischen Selbstversuch auf Code-Ebene untersucht und final in die Multi-Agenten-Architektur integriert werden:

5.1.1 Evaluation der funktionalen Praxisanforderungen

1. **Konsistenzprüfung im Testprozess (Out-of-Scope):** Der Wunsch nach einer automatisierten Überführung und Validierung historischer Testkonfigurationen stellt in realen SAP-Migrationsprojekten eine erhebliche Entlastung dar. Die softwaretechnische Umsetzung im Rahmen dieses agentischen Proof-of-Concepts wird jedoch als *Out-of-Scope* definiert. Die Migration von Test-Sets erfordert eine tiefe Kopplung an proprietäre und unternehmensspezifische Test-Infrastrukturen sowie unstrukturiertes Domänenwissen über Altsystemzustände. Da diese externe Umgebung im Rahmen des Prototyps nicht standardisiert und isoliert simuliert werden kann, ist eine verlässliche Validierung durch KI-Agenten ohne menschlichen Kontext unmöglich. Die hypothetische Rolle eines dedizierten „Tester-Agenten“ wird für diesen Prototyp somit verworfen.
2. **Semantische Analyse und Code-Bereinigung (In-Scope):** Die Identifikation von funktional obsoleten Altlasten (wie ungenutzten Merkmalen oder verwaistem Beziehungswissen) im Legacy-Code lässt sich direkt auf syntaktischer Ebene analysieren. Da diese Bereinigung die kognitive Last des Entwicklers senkt, wird sie in den Scope des Systems aufgenommen. Der praktische Selbstversuch wird dazu genutzt, wiederkehrende Muster von „totem Code“ zu identifizieren, um dem späteren System (konkret der Rolle des Analysten-Agenten) entsprechende Heuristiken mitzugeben.
3. **Automatisierte Logik-Transformation (In-Scope – Kernfokus):** Die empirische Baseline hat gezeigt, dass die manuelle Umstrukturierung und Neucodierung von Vorbedingungen und Prozeduren rund zwei

Drittel des Gesamtaufwands beansprucht. Diese Anforderung bildet den absoluten Kernbereich des zu konstruierenden Artefakts. Der Selbstversuch konzentriert sich maßgeblich auf diesen Flaschenhals, um die komplexen, nicht-trivialen Transformationspfade auf Code-Ebene physisch zu durchdringen und in formale Übersetzungsregeln zu überführen.

5.1.2 Evaluation der architektonischen Paradigmen

1. **Interaktive Assistenz / Consulting-Ansatz (Reduzierter Scope):** Ein vollumfänglicher, frei textender „Consultant-Agent“, der wie ein menschlicher Berater strategische Design-Entscheidungen diskutiert, übersteigt die Ressourcen und den deterministischen Fokus einer automatisierten Code-Transformation. Um den Praxisbedarf nach Transparenz dennoch zu erfüllen, wird dieser Ansatz auf eine *diagnostische Review-Ebene* reduziert. Das System wird so konzipiert, dass es im Falle von syntaktischen Unschärfen oder nicht eindeutigen Übersetzungen automatische Warnhinweise und Kommentare generiert (*Review-Flags*), anstatt autonome, potenziell fehlerhafte Entscheidungen zu treffen.
2. **Einhaltung von AVC-Modellierungsrichtlinien (In-Scope):** Die Erzeugung von standardkonformem und syntaktisch fehlerfreiem AVC-Zielcode ist für die Lauffähigkeit des Systems essenziell. Die Ersetzung von obsoleten Elementen (wie klassischen Z-Funktionen) durch native AVC-Standardfunktionen wird als feste methodische Leitplanke in das Systemdesign integriert. Der Selbstversuch dient hierbei als Werkzeug, um die genauen semantischen Äquivalente im AVC-Standard-Repository für den Legacy-Code zu ermitteln.

5.1.3 Synthese des finalen System-Scopes

Zusammenfassend führt dieser technologische Realitätsabgleich zu einer gezielten Fokussierung des Forschungsdesigns. Für den anschließenden praktischen Selbstversuch (vgl. Kapitel 5.2 ff.) bedeutet dies, dass die Stichprobenziehung (*Sampling*) und die qualitative Analyse ausschließlich auf die Bereiche der **Code-Bereinigung**, der **Logik-Transformation** (insb. Vorbedingungen und Prozeduren) sowie die Definition von **diagnostischen Warnhinweisen** ausgerichtet werden.

Für die darauffolgende Artefaktkonstruktion der Multi-Agenten-Architektur (Version 1) wird die ursprüngliche Rollenhypothese somit geschärft: Das System verzichtet auf autonome Test- und vollumfängliche Chatbot-Komponenten

und konzentriert sich stattdessen auf ein hocheffizientes, kollaboratives Kernteam, bestehend aus einem spezialisierten ****Analysten-Agenten**** (Bereinigung und Strukturierung) und einem ****Syntaktiker-Agenten**** (Code-Transformation und Diagnostik).

5.2 Wissenschaftliche Einordnung und methodisches Analyseprotokoll

Das methodische Vorgehen folgt den Richtlinien für explorative Fallstudien (*Exploratory Case Studies*) im Software Engineering nach Runeson und Höst (2009), kombiniert mit einer qualitativen Artefaktanalyse nach Seaman (1999). In der empirischen Softwaretechnik gelten Quellcode und Systemkonfigurationen als vollwertige, objektive Dokumente. Die qualitative Analyse dieser Artefakte ist notwendig, um unbewusstes Handeln und implizites Erfahrungswissen (*Tacit Knowledge*) von Entwicklern offenzulegen, welches in reinen Befragungen oft ungenannt bleibt (Seaman, 1999).

Der manuelle Migrationsprozess wird hierbei als Untersuchungsobjekt im realen Kontext der *msg systems ag* betrachtet, wobei der Autor die Rolle des transformierenden Entwicklers einnimmt. Um den Vorwurf einer unsystematischen Vorgehensweise auszuschließen, folgt die Analyse einem strukturierten, protokollbasierten Datenreduktions- und Codierungsverfahren in Anlehnung an die von Seaman (1999) postulierten Techniken zur qualitativen Strukturierung von Software-Artefakten. Dieses Protokoll gliedert sich in drei aufeinander aufbauende Phasen:

1. **Strukturelle Dekonstruktion (Input-Analyse):** Das systematische Sichten und Aufschlüsseln des originären, imperativen LO-VC-Codes. Die Logikbausteine werden theoriegeleitet isoliert und im Sinne des qualitativen Codings nach Seaman (1999) nach ihrer logischen Funktion (z. B. Wertezuweisung, Bedingungsprüfung, Tabellenaufruf) kategorisiert.
2. **Systematische Synthese (Target-Mapping):** Die manuelle, regelkonforme Überführung der isolierten Fachlogik in die deklarative Ziel-Syntax der *Advanced Variant Configuration* (AVC) unter strikter Einhaltung der aktuellen SAP-Modellierungsrichtlinien.
3. **Muster-Extraktion, Scoping und induktive Erweiterung:** Die vergleichende Gegenüberstellung von Input und Output zur Identifikation wiederkehrender Transformationsmuster (*Mapping-Rules*). Kritischer Bestandteil dieser Phase ist die qualitative Bewertung jedes Musters

auf seine Automatisierbarkeit hin. Gleichzeitig bleibt das Verfahren gemäß den Prinzipien von Runeson und Höst (2009) offen für *induktive Erkenntnisse* – also das ungeplante Entdecken neuer syntaktischer Komplexitäten oder Abhängigkeiten während des Migrationsaktes, die im Experteninterview noch nicht identifiziert wurden und den Scope des MAS maßgeblich beeinflussen.

Durch dieses protokollierte Vorgehen wird eine lückenlose Beweiskette (*Chain of Evidence*) etabliert: Von der empirischen Identifikation realer Code-Altlasten über die methodische Abgrenzung des System-Scopes bis hin zur Formulierung valider, praxistauglicher Verhaltensregeln für die spätere informationstechnische Konstruktion des Artefakts.

5.3 Datenbasis und Sampling-Strategie

Um die abgeleiteten Anforderungen physisch auf Code-Ebene zu evaluieren, bedarf es einer repräsentativen Datengrundlage. Aufgrund von strikten Restriktionen beim Datenzugriff, dem Schutz von Geschäftsgeheimnissen sowie der begrenzten Verfügbarkeit extrahierbarer Altdaten im Unternehmenskontext der msg systems ag, wird in dieser Arbeit auf das Verfahren der Gelegenheitsstichprobe (Convenience Sampling) zurückgegriffen (Seaman, 1999).

Als Basis diene ein isoliertes, reales Kundenprojekt, welches die typische strukturelle Komplexität der industriellen Variantenkonfiguration aufweist. Aus diesem Projekt wurden zufällig fünf unterschiedliche Merkmale und deren zugehöriges, historisch gewachsenes Beziehungswissen (Vorbedingungen, Prozeduren) extrahiert. Diese Methodik verzichtet bewusst auf eine künstliche Selektion von Extremfällen (Maximum Variation Sampling), sondern bildet stattdessen den alltäglichen Entwickleralltag ab. Diese Basis stellt sicher, dass die identifizierten kognitiven Hürden und die daraus abgeleiteten Systemregeln nicht auf theoretischen Ausnahmefällen, sondern auf der realen, durchschnittlichen Systemkomplexität von msg-Kundenprojekten beruhen.

5.4 Ergebnisse des Selbstversuchs: Ableitung der kognitiven Architektur

Die detaillierte Protokollierung der manuellen Migration für alle fünf untersuchten Logik-Artefakte – inklusive des rohen LO-VC-Input-Codes, der kognitiven Zwischenschritte sowie des finalen AVC-Zielcodes – befindet sich

aus Gründen der Lesbarkeit im Anhang ???. Im folgenden Abschnitt werden die aus dieser Fallstudie extrahierten zentralen Erkenntnisse und die daraus abgeleiteten systematischen Handlungsregeln für die Multi-Agenten-Architektur synthetisiert.

Die qualitative Auswertung des manuellen Migrationsprotokolls zeigt deutlich, dass eine erfolgreiche Transformation von LO-VC nach AVC weit über das einfache Ersetzen von Syntax-Token hinausgeht. Die protokollierten Hürden verdeutlichen, dass während der Migration unbewusst ein dreistufiges kognitives Modell durchlaufen wird. Eine simple 1:1-Übersetzung würde die moderne Gecode-Engine des AVC ineffizient nutzen oder zu fehlerhaftem Code führen. Aus den fünf untersuchten Artefakten lassen sich folgende zentrale kognitive Phasen ableiten, die als direkte Blaupause für das zu entwickelnde IT-Artefakt dienen:

1. **Strukturelle und semantische Bereinigung (Die Analysten-Perspektive)**

Bevor der eigentliche Code transformiert wird, muss der historische Kontext bereinigt werden. In den Artefakten zeigte sich wiederholt, dass historisch gewachsene Modelle oft projektspezifische Anti-Patterns und verwaiste Abhängigkeiten enthalten, die funktionslosen („toten“) Code markieren. Zudem war der Legacy-Code stark durch alte Entwicklerkommentare (mit * markiert) oder logische Redundanzen fragmentiert. Der erste notwendige Schritt ist folglich die rigorose Filterung dieser obsoleten Elemente, um die Datengrundlage für die nachfolgende Logik-Transformation zu bereinigen und die Fehleranfälligkeit zu senken.

2. **Architektonische Design-Entscheidungen (Die Architekten-Perspektive)**

Nach der Bereinigung der Rohdaten muss über die strukturelle Zielform entschieden werden. Die Fallstudie belegt, dass dies der komplexeste Schritt ist:

- Teilen sich diverse Einzelwerte exakt dieselben Vorbedingungen, ist es architektonisch ineffizient, diese in redundante Einzelanweisungen zu übersetzen. Stattdessen müssen diese Muster erkannt und zu performanten Variantentabellen geclustert (aggregiert) werden.
- Sind Variantentabellen aufgrund komplexer Ungleichungen nicht optimal, müssen alternative Bündelungen (etwa durch den IN-Operator) als Bauplan entworfen werden.
- Gleichzeitig muss das System erkennen, welche Strukturblocke gänzlich obsolet sind, da AVC beispielsweise den alten INFERENCES-Block in Constraints nicht mehr unterstützt.

3. Syntaktische Synthese (Die Entwickler-Perspektive)

Erst nach der Erstellung dieses logischen Bauplans erfolgt das eigentliche Programmieren unter strikter Einhaltung der rigiden AVC-Syntaxregeln. Hierzu gehört das formale Schreiben der Restriktionen, das Auflösen veralteter Blockstrukturen durch direkte Pfadreferenzierungen mittels `$self.` oder `$root.` sowie das rigorose Ersetzen veralteter relationaler Operatoren (z. B. `ne` durch `<>` oder `eq` durch `=`).

5.4.1 Ableitung der V1-Architektur und System-Prompts

Um diese menschliche kognitive Leistung maschinell nachzubilden, scheidet ein monolithischer LLM-Ansatz (Single-Prompting) aus. Ein einzelnes Sprachmodell wäre mit der gleichzeitigen Ausführung von Bereinigung, Architekturentscheidung und Syntax-Formatierung überfordert. Stattdessen diktieren die empirischen Ergebnisse dieses Selbstversuchs eine arbeitsteilige Multi-Agenten-Architektur.

Das im anschließenden Kapitel 6 konzipierte System wird exakt nach diesem dreistufigen Vorbild aufgebaut und umfasst ein kollaboratives Kernteam aus drei KI-Agenten, gestützt durch ein deterministisches Validierungswerkzeug. Aus der manuellen Fallstudie lassen sich hierfür folgende initiale Rollendefinitionen und Handlungsregeln (Prompts) für den Prototyp extrahieren:

Rolle 1: LO-VC Data Analyst & Cleaner

Ziel: Bereinigung der rohen JSON-Eingabedaten.

Handlungsregel: Analysiere komplexe Vorbedingungen, identifiziere und ignoriere Werte, die historisch gewachsenen toten Code (Anti-Patterns) enthalten. Extrahiere für die verbleibenden, gültigen Werte eine saubere Liste der steuernden Merkmale und entferne alte Code-Kommentare.

Rolle 2: AVC Table Architect

Ziel: Entwurf eines logischen Bauplans basierend auf den bereinigten Daten.

Handlungsregel: Entscheide, welche Merkmale die Spalten einer potenziellen Tabelle bilden. Identifiziere exakt identische Vorbedingungen und instruiere den nachfolgenden Agenten, diese effizient zu aggregieren (z. B. mittels `IN()`-Operator oder Variantentabellen).

Rolle 3: AVC Syntax Coder (Synthesizer)

Ziel: Übersetzung des Bauplans in fehlerfreie, strikte AVC-Syntax.

KAPITEL 5. DURCHFÜHRUNG UND AUSWERTUNG DES PRAKTISCHEN SELBSTVERSUS

Handlungsregel: Schreibe den finalen Code (TABLE-Aufrufe und RESTRICT-Constraints). Nutze zwingend korrekte Objektreferenzen (z. B. `$self.`) und übersetze alte Operatoren (z. B. `eq`, `ne`) in den aktuellen Standard.

Tool-Integration:

Übergib das Ergebnis zwingend an einen deterministischen Syntax- & Format-Validator, der den Code programmatisch auf verbotene LO-VC-Relikte prüft und bei Fehlern eine Korrekturschleife erzwingt.

Diese empirisch hergeleitete Agenten-Konstellation bildet das logische Fundament für die softwaretechnische Implementierung des Design Science Artefakts.

Kapitel 6

Systemdesign und Prototypenentwicklung

Nachdem im vorangegangenen Abschnitt die logische Zielarchitektur des Multi-Agenten-Systems konzeptionell aus der Anforderungsanalyse hergeleitet wurde, dokumentiert dieses Kapitel die informationstechnische Umsetzung des Prototyps. Die Entwicklung gliedert sich hierbei in zwei Phasen: Zunächst wird die Technologieauswahl und Infrastruktur begründet, um die bei diesem Projekt relevanten Enterprise-Anforderungen abzusichern. Darauf aufbauend wird im Sinne des *Design Science Research* (DSR) die iterative, evolutionäre Entwicklung der Systemarchitektur dokumentiert, welche durch kontinuierliche Evaluierungszyklen zur finalen Pipeline führte.

6.1 Technologieauswahl und Infrastruktur

Die Auswahl der genutzten Frameworks und Plattformen richtet sich nach den internen Vorgaben der *msg systems ag* an Datenschutz und IT-Sicherheit sowie nach grundlegenden Prinzipien der Softwareentwicklung wie Wartbarkeit und Flexibilität. Das technische Fundament des Prototyps besteht aus drei Komponenten: dem Orchestrierungs-Framework *CrewAI*, dem *SAP AI Core* als sicherer Ausführungsumgebung und *liteLLM* als standardisierter Schnittstelle zur Modellanbindung.

6.1.1 Orchestrierungs-Framework: CrewAI

Laut Liu et al. (2025) gehören LangGraph, AutoGen und CrewAI zu den aktuell etabliertesten Frameworks für Multi-Agenten-Systeme. Für den Prototyp in dieser Arbeit fiel die Wahl auf *CrewAI*. Diese Entscheidung basiert

auf den strukturellen Anforderungen der Code-Migration und dem Ziel, die Entwicklungszeit für den Prototyp effizient zu halten.

Die Alternativen weisen für diesen speziellen Anwendungsfall konzeptionelle Nachteile auf: *AutoGen* ist primär auf offene, konversationelle Dialoge zwischen Agenten ausgelegt, was für starre Übersetzungsprozesse weniger geeignet ist. *LangGraph* unterstützt zwar deterministische Workflows, erfordert jedoch einen vergleichsweise hohen Programmieraufwand bei der Definition der einzelnen Prozesszustände. *CrewAI* nutzt stattdessen einen deklarativen, rollenbasierten Ansatz, der den nötigen Code für die Orchestrierung der Agenten deutlich reduziert (Singh, 2025). Dies beschleunigt die Implementierung und eignet sich daher besonders für die Prototypenentwicklung (Singh, 2025).

Ein weiterer Vorteil von *CrewAI* für dieses Projekt ist die sequenzielle Struktur des Frameworks, die bei der Agenten-Koordination zu geringen Latenzzeiten führt (Singh, 2025). Die Übersetzung von LO-VC nach AVC ist ein strikt aufeinander aufbauender Prozess, bestehend aus Analyse, Architekturentwurf und Code-Synthese. *CrewAI* ermöglicht es, diese Reihenfolge als feste Pipeline abzubilden. Dadurch wird sichergestellt, dass die Agenten ihre Aufgaben strukturiert nacheinander abarbeiten. Diese Einschränkung der Freiheitsgrade ist bei der Code-Generierung gewünscht, da sie das Risiko von Fehlern durch ungeplante Agenten-Interaktionen minimiert.

6.1.2 Infrastruktur und Modellintegration: SAP AI Core und liteLLM

Bei der automatisierten Verarbeitung von industriellen Produktdaten und Konfigurationsmodellen (LO-VC) müssen interne Datenschutzvorgaben zwingend eingehalten werden. Da diese Modelle vertrauliches Firmenwissen enthalten, ist die direkte Übertragung an öffentliche APIs (wie die Standard-Schnittstelle von OpenAI) aus Sicherheitsgründen ausgeschlossen.

Um diese Vorgaben zu erfüllen, erfolgt die Anbindung der Sprachmodelle in diesem Prototyp über den *SAP AI Core*. Dies stellt sicher, dass die Datenverarbeitung DSGVO-konform in einer kontrollierten Systemumgebung stattfindet. Neben dem Datenschutz hat diese Entscheidung auch organisatorische Gründe: Da die *msg systems ag* den *SAP AI Core* bereits in anderen Projekten einsetzt, ist die nötige Infrastruktur bereits vorhanden. Dies erleichtert die technische Bereitstellung und ermöglicht es, die anfallenden Nutzungskosten standardisiert über bestehende Unternehmensverträge abzurechnen.

Ein weiterer technischer Aspekt betrifft die Anbindung der Modelle an das Agentensystem. Um flexibel auf technologische Änderungen reagieren

zu können, soll der Prototyp nicht fest an einen einzelnen Anbieter oder ein spezifisches Modell gebunden werden (*Vendor Lock-in*).

Um dies zu verhindern, wird die Python-Bibliothek *liteLLM* genutzt und in das *CrewAI*-Framework integriert. Sie fungiert als standardisierende Schnittstelle zwischen den KI-Agenten und den Sprachmodellen. Dadurch wird der Quellcode der Multi-Agenten-Architektur vom verwendeten Modell entkoppelt. Das ermöglicht es, das zugrundeliegende Modell bei Bedarf mit minimalem Konfigurationsaufwand über den *SAP AI Core* auszutauschen, ohne den Programmcode der eigentlichen Agenten anpassen zu müssen.

6.2 Iterative Entwicklung des Prototyps (Design Science Research)

Gemäß dem *Design Science Research*-Ansatz nach Hevner et al. wurde das Multi-Agenten-System in iterativen Entwicklungs- und Testzyklen (*Build-and-Evaluate*) umgesetzt. Um die Ergebnisse objektiv vergleichen zu können, wurde in der gesamten Prototyping-Phase eine konstante Datenbasis verwendet. Diese bestand aus fünf Beispielsmerkmalen inklusive ihrer Ausprägungen und Vorbedingungen, die aus einem realen Kundenprojekt extrahiert wurden.

Die folgenden Abschnitte dokumentieren die schrittweise Weiterentwicklung der Systemarchitektur. In jeder Phase wurden bestehende Fehler und technische Limitierungen analysiert und behoben, bis die finale Architektur feststand.

6.2.1 Zyklus 1: Limitierungen einer rein agentenbasierten Architektur

Die erste Version des Prototyps (V1-Architektur) basierte auf der Annahme, dass ein Netzwerk aus drei KI-Agenten (Analyst, Architekt, Synthesizer) die JSON-Daten schrittweise verarbeiten kann. Der Prozess lief sequenziell pro Merkmal ab: Das System erhielt ein standardisiertes JSON-Dokument mit allen Merkmalen, Werten und Vorbedingungen. Die Agenten sollten die Daten nacheinander bearbeiten und am Ende ein strukturiertes Zielformat ausgeben. Dieses Ergebnis sollte entweder aus einfachem AVC-Code bestehen (falls das System entscheidet, keine Tabelle zu generieren) oder aus einem Tabellenvorschlag inklusive des zugehörigen AVC-Aufrufs.

Die ersten Tests mit dieser grundlegenden Architektur zeigten jedoch deutliche Schwächen:

- **Mangelnde Reproduzierbarkeit:** Das System lieferte keine deterministischen Ergebnisse. Bei drei Durchläufen mit exakt derselben Datenbasis generierten die Agenten drei unterschiedliche Ausgaben.
- **Fehlendes Domänenwissen:** Einem *Large Language Model* (LLM) lediglich per System-Prompt die Rolle eines SAP-AVC-Entwicklers zuzuweisen, reichte nicht aus. Das Modell kannte die spezifische Syntax nicht ausreichend, verlor schnell den fachlichen Kontext und nutzte falsche Bezeichnungen für Merkmale und Werte.
- **Probleme bei der Tabellenbildung:** Die Hauptaufgabe – zu bewerten, wann eine Variantentabelle sinnvoll ist und wann nicht – überforderte das System. Besonders in Fällen, in denen innerhalb eines Merkmals nur bestimmte Werte für eine Tabelle geeignet waren und andere isoliert behandelt werden mussten, konnte das LLM keine korrekten und konsistenten Entscheidungen treffen.

6.2.2 Zyklus 2: Code-Refaktorisierung, Wissensbasis und strukturelle Halluzinationen

Um den identifizierten Schwächen aus dem ersten Zyklus zu begegnen, wurden in der zweiten Iteration mehrere methodische und strukturelle Anpassungen am Prototyp vorgenommen. Zunächst wurde der Quellcode in verschiedene Module (`main.py`, `agents.py`, `utils.py`) aufgeteilt, um die Übersichtlichkeit und Wartbarkeit der Anwendung zu verbessern. Zudem wurden die Aufgabenbeschreibungen (Prompts) der einzelnen Agenten inhaltlich deutlich geschärft.

Um das im ersten Durchlauf fehlende Syntaxwissen auszugleichen, wurde ein separates Wissensdokument angelegt, das dem Synthesizer-Agenten als Referenz für gültigen AVC-Code dienen sollte. Zur besseren Überprüfbarkeit der generierten Ergebnisse wurde außerdem eine Hilfsfunktion in die `utils.py` integriert. Diese formatiert den Text-Output des *Large Language Models* so um, dass nach jedem Durchlauf automatisch ein strukturiertes Excel-Dashboard erstellt wird.

Die Evaluierung dieses angepassten Systems zeigte zunächst einen organisatorischen Erfolg: Die Aufbereitung der Ergebnisse im Excel-Dashboard funktionierte zuverlässig und erleichterte die manuelle Code-Prüfung erheblich. Bei der inhaltlichen Logikgenerierung traten jedoch neue Probleme auf:

- **Anhaltende Stochastik:** Trotz der geschärften Prompts blieb das System unzuverlässig. Drei identische Durchläufe produzierten weiterhin drei unterschiedliche Ergebnisse.

- **Übergenerierung und Halluzinationen:** Das System verstand nun zwar grundsätzlich den Auftrag, zwischen Variantentabellen und isolierten *IF*-Bedingungen (Standalone) zu unterscheiden. Allerdings zeigte das LLM die Tendenz, zwanghaft für jedes Merkmal riesige Tabellen generieren zu wollen, in die alle Werte gequetscht wurden. Dabei verlor das Modell den Bezug zum JSON-Input: Es erfand Spaltennamen (teilweise aus einfachen Syntax-Fragmenten), halluzinierte nicht-existente Tabelleneinträge und behalf sich mit Platzhaltern, um die fehlerhaften Tabellenstrukturen künstlich aufrechtzuerhalten.
- **Problem des festverdrahteten Domänenwissens:** Ein konzeptioneller Fehler zeigte sich beim Umgang mit spezifischen Kundendaten. Das Herausfiltern des Wertes „specified dummy“ (ein kundenindividueller Workaround für obsoleten Code) war in dieser Phase noch fest im System-Prompt des Agenten einprogrammiert (Hardcoding).

Um die mangelnde Flexibilität beim Domänenwissen sofort zu adressieren, wurde noch innerhalb dieses Zyklus ein externes Konfigurationsdokument eingeführt. Der Entwickler kann hier projektbezogene Sonderregeln dynamisch hinterlegen, wodurch das Kernsystem domänenunabhängig wird. Begleitend wurden die System-Prompts der Agenten sowie das Dokument für die AVC-Syntax weiter detailliert, in der Hoffnung, die Präzision der Tabellengenerierung zu erhöhen.

Die erneute Prüfung dieser spezifischen Anpassungen brachte jedoch keine signifikante Verbesserung. Die Ergebnisse blieben bei mehrfacher Ausführung weiterhin inkonsistent, waren fachlich fehlerhaft und von den beschriebenen strukturellen Halluzinationen geprägt.

Die abschließende Erkenntnis aus Zyklus 2 zeigte unmissverständlich: Ein Sprachmodell ist selbst mit ausgelagertem Domänenwissen, externen Hilfsdokumenten und maximal geschärften Rollen nicht in der Lage, deterministische Datenstrukturen (wie Tabellen) konsistent und fehlerfrei aus einem JSON zu extrahieren.

6.2.3 Zyklus 3: Regelbasierte Vorverarbeitung und Domänenunabhängigkeit

Die Tests zeigten, dass die architektonische Entscheidungsfindung (Tabelle vs. Einzelbedingung) für das Sprachmodell zu komplex war. Um die Aufgabe zu vereinfachen, musste geklärt werden, ob es für die Erstellung von Variantentabellen klare, deterministische Regeln gibt. Die Rücksprache mit einem Fachexperten bestätigte dies: Eine Tabelle ist in der industriellen Praxis

nur dann sinnvoll, wenn direkte Gleichsetzungsbedingungen vorliegen (z. B. `Merkmal_A = '1'`). Bei Ungleichungen (z. B. `<>`) verbietet sich eine Tabelle. Zudem lohnt sich der Aufbau einer Tabelle meist erst, wenn mehr als zwei Merkmale involviert sind, um eine Fragmentierung in viele Kleinsttabellen zu vermeiden.

Ein zweiter wichtiger Punkt aus dem Experteninterview betraf den Umgang mit der `specified`-Syntax. Diese prüft, ob ein Merkmal im Material überhaupt bewertet wurde. Der Kunde nutzte dies als Workaround, um obsoleten Code (Dummy-Werte) zu blockieren, da diese Dummy-Merkmale im Material nie klassifiziert waren. Um dieses harte Domänenwissen nicht mehr im Agenten-Prompt festschreiben zu müssen, wurde ein vorgeschaltetes Python-Skript entwickelt. Dieses Skript liest zunächst das gesamte JSON-Dokument ein und erstellt eine vollständige Liste aller im Material existierenden Merkmale. Mit dieser Liste kann das System im Vorfeld deterministisch prüfen, ob ein Ausdruck wie `specified(Merkmal_X)` wahr oder falsch ist, und den entsprechenden Code-Block automatisch verarbeiten oder ignorieren.

Zur besseren Fehleranalyse wurden ab dieser Phase zudem dedizierte Debugging-Dateien generiert, um die Zwischenergebnisse der Vorverarbeitung gezielt prüfen zu können, ohne immer den finalen Code abwarten zu müssen. Eines dieser Zwischenergebnisse war das gezielte Markieren von unklassifizierten Merkmalen durch den Python-Filter, um dem Agenten die syntaktische Verarbeitung zu erleichtern.

Die Evaluierung dieses Zyklus zeigte spürbare Verbesserungen: Die Ergebnisse wirkten weniger zufällig, und die Trennung zwischen Tabellenvorschlägen und *IF*-Bedingungen funktionierte teilweise. Dennoch traten bei mehrfachen Durchläufen weiterhin Halluzinationen und Inkonsistenzen auf. Zudem zeigte sich eine neue strukturelle Herausforderung: Das System erzeugte redundante Tabellen. Teilweise waren neu generierte Tabellen exakte Duplikate, teilweise handelte es sich um logische Teiltabellen (Sub-Tabellen), die inhaltlich bereits durch größere Tabellen abgedeckt waren.

6.2.4 Zyklus 4: Die finale neuro-symbolische Architektur und funktionale Abgrenzung

Aufgrund der begrenzten Bearbeitungszeit der Masterarbeit mussten in Bezug auf diese Tabellen-Redundanzen und die Syntax-Prüfung klare Grenzen für den Funktionsumfang (Scope) des Prototyps gezogen werden:

- **Tabellen-Redundanzen:** Das Erkennen und Eliminieren von logischen Teiltabellen erfordert komplexe mathematische Berechnungen (z. B.

durch einen SAT-Solver). Da dies zeitlich nicht umsetzbar war, verbleibt dieses Problem in der *Future Work*. Umgesetzt wurde lediglich ein Modul, das exakt identische Tabellen zusammenführt (Join). Obwohl auch dies deterministisch über Python lösbar wäre, wurde es aus Zeitgründen als Agenten-Aufgabe im System belassen.

- **Syntax-Validierung:** Eine fehlerfreie syntaktische Prüfung von AVC-Code erfordert idealerweise einen *Abstract Syntax Tree* (AST) Parser. Auch dies wurde in die *Future Work* verschoben. Die Validierung der häufigsten Fehler (z. B. fehlende `$self`-Präfixe) übernimmt im finalen Prototyp stattdessen ein dedizierter KI-Agent mit rudimentären Prüfregeln.

Die weitreichendste Änderung in diesem finalen Zyklus war die massive Ausweitung der deterministischen Vorverarbeitung (Preprocessing). Es wurde ein umfassendes Python-Skript integriert, das den Agenten den Großteil der kognitiven Arbeit abnimmt: Es arbeitet auf einer Kopie der Rohdaten, löscht leere Werte ohne Vorbedingungen und bereinigt alte Kommentare. Es verknüpft mehrere Vorbedingungen logisch mit **AND**, korrigiert fehlerhafte Altlogik („Ghost Logic“), markiert unklassifizierte Merkmale und fällt vor allem die strikte, deterministische Entscheidung, ob und wie viele Tabellen für ein Merkmal gebaut werden. Zuletzt wurde eine Hilfsfunktion implementiert, die sicherstellt, dass aus der Agenten-Antwort ausschließlich reines JSON extrahiert wird, um Programmabstürze durch unnötigen Begleittext (z. B. „Hier ist das Ergebnis:“) zu verhindern.

Ergebnis und architektonisches Fazit:

Durch diese Anpassungen entwickelte sich die Architektur von einer reinen KI-Pipeline zu einem *neuro-symbolischen* System. Die Künstliche Intelligenz wurde schrittweise aus der logischen Entscheidungsfindung verdrängt. Dem LLM verbleibt im finalen Prototyp lediglich die Aufgabe, den vorbereiteten Bauplan in AVC-Code zusammenzusetzen und grundlegend zu validieren.

Die Ergebnisse waren im letzten Testlauf reproduzierbar; das System lieferte bei drei Durchläufen dreimal exakt dasselbe Resultat. Die zentrale Erkenntnis dieser iterativen Entwicklung ist, dass der Problemraum der Code-Migration stark deterministisch ist. Der Einsatz von generativer KI erweist sich für die strukturelle Analyse als ineffizient und ist bei vollständiger Umsetzung mathematischer Solver (wie AST-Parser und SAT-Solver) für diesen Teilprozess weitestgehend obsolet.

Literatur

- Blumöhr, U., Kölbl, A., Neuhaus, M., & Ukalovic, M. (2023). *Advanced Variant Configuration in SAP S/4HANA*. Rheinwerk Publishing. Verfügbar 6. Mai 2026 unter <https://katalog.ub.uni-heidelberg.de/titel/69115224>
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., & Askell, A. (2020). Language Models Are Few-Shot Learners. *Advances in neural information processing systems*, 33, 1877–1901. Verfügbar 16. Mai 2026 unter https://proceedings.neurips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html?utm_source=transaction&utm_medium=email&utm_campaign=linkedin_newsletter
- Chase, H. (2024). *Langchain*. <https://github.com/langchain-ai/langchain>
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021, 14. Juli). *Evaluating Large Language Models Trained on Code*. arXiv: 2107.03374 [cs]. <https://doi.org/10.48550/arXiv.2107.03374>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. *MIS quarterly*, 28(1), 75–106. Verfügbar 20. Juni 2026 unter <https://misq.umn.edu/misq/article/28/1/75/261>
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Yau, S., Lin, Z., & Zhou, L. (2024). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *International Conference on Learning Representations, 2024*, 23247–23275. Verfügbar 20. Mai 2026 unter https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html
- Hove, S. E., & Anda, B. (2005). Experiences from Conducting Semi-Structured Interviews in Empirical Software Engineering Research. *11th IEEE International Software Metrics Symposium (METRICS'05)*, 10–pp.

- Verfügbar 31. März 2026 unter <https://ieeexplore.ieee.org/abstract/document/1509301/>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). Swe-Bench: Can Language Models Resolve Real-World Github Issues? *International Conference on Learning Representations, 2024*, 54107–54157. Verfügbar 20. Mai 2026 unter https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- Karabiyik, M. A. (2025). RefactorGPT: A ChatGPT-based Multi-Agent Framework for Automated Code Refactoring. *PeerJ Computer Science*, 11, e3257. Verfügbar 21. April 2026 unter <https://peerj.com/articles/cs-3257/>
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., & Ghanem, B. (2023). Camel: Communicative Agents forMMindExploration of Large Language Model Society. *Advances in neural information processing systems*, 36, 51991–52008. Verfügbar 20. Mai 2026 unter <https://proceedings.neurips.cc/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html>
- Liu, D., Upadhyay, K., Chhetri, V., Siddique, A. B., & Farooq, U. (2025). A Large-Scale Study on the Development and Issues of Multi-Agent AI Systems. *2025 IEEE International Conference on Big Data (BigData)*, 7785–7792. Verfügbar 2. Juni 2026 unter <https://ieeexplore.ieee.org/abstract/document/11402104/>
- Matilainen, A. (2024). Change Requirements on Product Data Structures in S/4HANA Implementation. Verfügbar 21. April 2026 unter <https://osuva.uwasa.fi/items/8bf7ae52-e756-4fca-9a25-c0510c58ad34>
- Mayring, P. (2014). Qualitative Content Analysis: Theoretical Foundation, Basic Procedures and Software Solution. Verfügbar 7. April 2026 unter https://www.ssoar.info/ssoar/bitstream/handle/document/39517/ssoar-2014-mayring-Qualitative_content_analysis_theoretical_foundation.pdf
- Moti, Z., Soudani, H., & van der Kogel, J. (2026, 14. März). *LegacyTranslate: LLM-based Multi-Agent Method for Legacy Code Translation*. arXiv: 2603.14054 [cs]. <https://doi.org/10.48550/arXiv.2603.14054>
- Oueslati, K., Lamothe, M., & Khomh, F. (2026, 4. März). *RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring*. arXiv: 2511.03153 [cs]. <https://doi.org/10.48550/arXiv.2511.03153>
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., & Cong, X. (2024). Chatdev: Communicative Agents for Software Development. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*,

- 15174–15186. Verfügbar 20. Mai 2026 unter <https://aclanthology.org/2024.acl-long.810/>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Russell, S. J. (2010). *Artificial Intelligence a Modern Approach*. Pearson Education, Inc. Verfügbar 26. Mai 2026 unter <https://scholar.alaqsa.edu.ps/9195/1/Artificial%20Intelligence%20A%20Modern%20Approach%20%283rd%20Edition%29.pdf%20%28%20PDFDrive%20%29.pdf>
- Seaman, C. B. (1999). Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on software engineering*, 25(4), 557–572. Verfügbar 31. März 2026 unter <https://ieeexplore.ieee.org/abstract/document/799955/>
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflection: Language Agents with Verbal Reinforcement Learning. *Advances in neural information processing systems*, 36, 8634–8652. Verfügbar 20. Mai 2026 unter https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in neural information processing systems*, 30. Verfügbar 16. Mai 2026 unter <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., & Liu, J. (2024). Autogen: Enabling next-Gen LLM Applications via Multi-Agent Conversations. *First Conference on Language Modeling*. Verfügbar 20. Mai 2026 unter https://openreview.net/forum?id=BAakY1hNKS&utm_source=
- Xu, Y., Lin, F., Yang, J., Tse-Hsun, Chen & Tsantalis, N. (2025, 27. März). *MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration*. arXiv: 2503.14340 [cs]. <https://doi.org/10.48550/arXiv.2503.14340>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *React: Synergizing Reasoning and Acting in Language Models*. Verfügbar 20. Mai 2026 unter <https://arxiv.org/abs/2210.03629>

- Zheng, Z., Ning, K., Wang, Y., Zhang, J., Zheng, D., Ye, M., & Chen, J. (2023). *A Survey of Large Language Models for Code: Evolution, Benchmarking, and Future Trends*. Verfügbar 14. Mai 2026 unter <https://arxiv.org/abs/2311.10372>

Anhang

Im Rahmen dieser Arbeit wurden folgende ergänzende Unterlagen erstellt:

- **Anhang A:** Datenauswertung der Experteninterviews (Excel)
- **Anhang B:** Leitfaden für die Befragung

Datenauswertung der Experteninterviews

Category Title	Marked Text
03_01_Kognitiv_Altlasten	In der LOVC-Welt war das etwas anders: Da konnten Merkmale irgendwo enthalten sein, auch wenn sie im Modell gerade nicht verwendet wurden.
03_01_Kognitiv_Altlasten	Wenn man ein Modell direkt konvertieren will, hat man oft Altlasten, die das Modell behindern, weil beispielsweise die Prozedur oder das Beziehungswissen nicht ausgeführt werden.
03_01_Kognitiv_Altlasten	Die Werteräume dieser Merkmale wachsen über die Zeit stetig an.
03_02_Kognitiv_Syntax_Umdenken	Viele Kunden haben sich zum Beispiel eigene Funktionen (Z-Functions) programmiert, die im Beziehungswissen aufgerufen werden.
03_02_Kognitiv_Syntax_Umdenken	Diese werden jetzt nicht mehr unterstützt.
03_02_Kognitiv_Syntax_Umdenken	Es gibt zwar BAdIs, mit denen wir einiges implementieren können, aber in der Clean-Core-Welt, in der alle Kunden jetzt unterwegs sein wollen, versucht man so etwas zu vermeiden.
03_02_Kognitiv_Syntax_Umdenken	Ein weiterer Punkt ist die von SAP empfohlene Modellierungstechnik für den AVC, die sich grundlegend von der im LOVC unterscheidet.
03_02_Kognitiv_Syntax_Umdenken	Diese sollen wir im AVC nun nicht mehr nutzen, sondern stattdessen mit Constraints arbeiten.
03_02_Kognitiv_Syntax_Umdenken	Man kann nichts direkt übernehmen. Auch wenn die Syntax eins zu eins gleich bleibt, muss ein neues Objekt generiert werden, da in den Eigenschaften des Objekts hinterlegt ist, ob es sich um klassisches Beziehungswissen oder um AVC handelt.
03_02_Kognitiv_Syntax_Umdenken	Der LOVC kommt mit nicht klassifizierten Merkmalen klar, der AVC hingegen nicht.
03_02_Kognitiv_Syntax_Umdenken	AVC an gewissen Stellen anders verhält als der LOVC.
03_02_Kognitiv_Syntax_Umdenken	Systemverhalten hat sich leicht geändert. Dieses veränderte Verhalten macht es oft notwendig, den Code anders zu schreiben oder um Dinge zu ergänzen, die vorher nicht da waren.
03_02_Kognitiv_Syntax_Umdenken	Das ist ein grundlegender Unterschied im Systemverhalten, der zu abweichenden Konfigurationsergebnissen führen kann und den man bei der Migration berücksichtigen muss.
03_02_Kognitiv_Syntax_Umdenken	Im AVC kann ich das nicht mehr spezifisch definieren. Im AVC werden immer alle Merkmale zueinander eingeschränkt.
03_02_Kognitiv_Syntax_Umdenken	Wenn ich also genau dasselbe Constraint wie vorher nun im AVC aufrufe, resultiert daraus möglicherweise ein ganz anderes Systemverhalten.
03_02_Kognitiv_Syntax_Umdenken	Das ist ein extremer Performance-Killer, den man laut SAP vermeiden soll.
03_03_Kognitiv_Fehler_Menschlich	Und am Ende war die Datei trotzdem fehlerhaft.
04_01_Tool_Konkrete_Anforderung	Man könnte sich vorstellen, diesen Ergebnisvergleich zu automatisieren oder von einer KI durchführen zu lassen.

04_01_Tool_Konkrete_Anforderung	Die KI könnte hierbei vielleicht noch bestimmte Hürden der Testautomatisierung überwinden.
04_01_Tool_Konkrete_Anforderung	Vielleicht könnte ein KI-Bot sich die alten Aufträge schnappen und aus den drei Einzelaufträgen automatisch ein gebündeltes Set bilden.
04_01_Tool_Konkrete_Anforderung	Man könnte beispielsweise einen Parser entwickeln, der die Syntax anhand fester Regeln übersetzt. Die KI sehe ich mittlerweile eher auf einer höheren Flugebene als beratendes Tool, das Zusammenhänge aufzeigt und Verbesserungspotenziale im Modell identifiziert.
04_01_Tool_Konkrete_Anforderung	Wenn ich ein Analysetool hätte, das so etwas erkennt, wäre das toll.
04_01_Tool_Konkrete_Anforderung	Die KI könnte so etwas erkennen und Ratschläge geben, wie: "Du machst die pauschale Bewertung am Anfang, setze sie doch ans Ende" oder "Du hast hier x Überschreibungen, die du vermeiden könntest, wenn du die Bedingungsteile präziser einschränkst, damit Sonderfall B nicht mehr im Sonderfall A enthalten ist."
04_01_Tool_Konkrete_Anforderung	Die KI könnte dann feststellen: "Die Prozedur an Position 10 wird im Kontext der anderen Prozeduren nie relevant sein, du kannst sie herausnehmen oder musst sie anders einsortieren."
04_01_Tool_Konkrete_Anforderung	dass die KI auch weiterhin zum Übersetzen genutzt wird.
04_01_Tool_Konkrete_Anforderung	fände ich es aber sehr gut, wenn du diese beratende Flugebene in dein Tool integrieren könntest.
04_01_Tool_Konkrete_Anforderung	wenn ich als Nutzer die KI zu meinem aktuellen Modell oder einer Funktion befragen könnte.
04_01_Tool_Konkrete_Anforderung	Die KI könnte beispielsweise den vorhandenen Code kommentieren.
04_01_Tool_Konkrete_Anforderung	Eine KI könnte hier automatisch Kommentare generieren, weil sie die Merkmale und die zugehörigen Beschreibungstexte miteinander verknüpft.
04_01_Tool_Konkrete_Anforderung	Es wäre cool, wenn eine KI in diese Z-Functions hineinschauen und analysieren könnte, was diese Funktion genau macht. Darauf aufbauend könnte sie vorschlagen, wie man diese Funktion im neuen System über ein BAdI lösen könnte.
04_01_Tool_Konkrete_Anforderung	Was extrem wertvoll wäre, ist, wenn die KI den Schritt von Vorbedingungen hin zu Constraints übernehmen könnte.
01_01_Prozess_Schritt_Manuell	Also musste der Mitarbeiter von Hand Bewertungen für die drei Modelle vornehmen, damit am Ende alles zusammenpasst
01_01_Prozess_Schritt_Manuell	Meistens schränke ich dort in Tabellenform die Werte zueinander ein und definiere gültige Kombinationen von Merkmalswerten.
01_01_Prozess_Schritt_Manuell	Man lädt sein LOVC-Modell dort hinein und kann es dann auf die Eignung für den AVC analysieren.
01_01_Prozess_Schritt_Manuell	Das ist quasi der erste Schritt: Man bekommt seine To-Dos aufgezeigt, also was im Beziehungswissen angepasst werden muss, und das Tool übernimmt die Massenanlage des Beziehungswissens.

01_01_Prozess_Schritt_Manuell	kann es anschließend aufräumen.
01_01_Prozess_Schritt_Manuell	Dafür muss ich die ganzen Vorbedingungen, die an den Werten hängen, analysieren und auswerten, was sie aussagen.
01_01_Prozess_Schritt_Manuell	Dort erstelle ich dann ein tabellenbasiertes Beziehungswissen für das jeweilige Produkt,
01_01_Prozess_Schritt_Manuell	erst einmal völlig losgelöst vom vorhandenen Konfigurationsmodell aus einer neutralen Sicht verstehen.
01_01_Prozess_Schritt_Manuell	fände ich es aber sehr gut, wenn du diese beratende Flugebene in dein Tool integrieren könntest.
01_01_Prozess_Schritt_Manuell	Dann habe ich geprüft, ob sich das deckt oder ob ich einen Denkfehler habe.
01_02_Prozess_Toolbruch	Meistens baue ich mir dafür eine große Excel-Tabelle auf.
02_01_Zeit_Dauer_Konkret	Sie haben ein einfaches bis mittleres Modell in drei bis vier Tagen migriert.
02_01_Zeit_Dauer_Konkret	Ich würde sagen, das macht etwa 20 % der Arbeit aus, der Großteil entfällt auf die Wandlung der Vorbedingungen in Constraints.
02_01_Zeit_Dauer_Konkret	20 % für die Merkmalswelt, 40 % für die Analyse und den Excel-Aufbau und der Rest entfällt auf die eigentliche Umsetzung.
02_01_Zeit_Dauer_Konkret	ein Kollege hat dort vier Monate lang in Vollzeit eine Excel-Tabelle aufgebaut, nur um die Vorbedingungen eines einzigen Modells zu analysieren.
02_01_Zeit_Dauer_Konkret	Diese Vorbedingungswelt durch constraint-basierte Modellierung abzulösen, ist eigentlich die Hauptarbeit bei dieser Migration.
02_02_Zeit_Flaschenhals	Ein Großteil der Arbeit besteht dabei darin, die vorab genannten Vorbedingungen durch Constraints abzulösen.
02_02_Zeit_Flaschenhals	Dieser Transfer also syntaktische Vorbedingungen in eine Tabellenform zu bringen und diese Tabellen dann in Constraints umzuwandeln macht ungefähr zwei Drittel der Arbeit aus.
02_02_Zeit_Flaschenhals	der Großteil entfällt auf die Wandlung der Vorbedingungen in Constraints.
02_02_Zeit_Flaschenhals	zu großen Teilen wird es dann zu einer reinen Umsetzungsaufgabe.
02_02_Zeit_Flaschenhals	20 % für die Merkmalswelt, 40 % für die Analyse und den Excel-Aufbau und der Rest entfällt auf die eigentliche Umsetzung.
Explizite_Tool_Nennung	Wenn man sich für eine direktere Migration entscheidet, gibt es von SAP mittlerweile ein Migration Cockpit, das beim Umstieg hilft.
Explizite_Tool_Nennung	im Trace
Testen	Es ist rein manuelles Testen.
Testen	durchaus Strategien für automatisches Testen.
Testen	letztes Jahr für den Kunden ein Konzept für ein mögliches Testtool geschrieben.
Testen	Der Prozess sieht so aus: Man nimmt sich einen vorhandenen Testauftrag als Basis und nutzt eine Migrationstabelle für die Merkmale.
Testen	Ich weiß also, was aus dem alten Merkmal A in meinem neuen Modell geworden ist beispielsweise Merkmal B. Damit gehe ich in die Konfiguration und lege einen Kundenauftrag im neuen System mit den neuen Merkmalsbewertungen an. Den Auftrag speichere ich ab und vergleiche dann den alten Kundenauftrag mit dem neuen.

Testen	Wir haben also die Konstellation, dass aus drei alten Objekten ein neues Objekt geworden ist. Diesen Schritt konnte das geplante Testtool bisher nicht abbilden.
Testen	Das Tool kann zwar das Fahrzeug in ein Testobjekt migrieren, aber die Batterie müsste von Hand nachkonfiguriert werden.
Komplexität_Modellgröße	Bei kleinen Modellen, wie die Kollegen sie aktuell betreuen, gibt es teilweise nur etwa 20 bis 30 Merkmale. Dort ist eine direkte Migration natürlich deutlich leichter möglich. haben wir letztes Jahr einen Proof of Concept für das Migrate-Tool gemacht. Dabei sind wir bei der reinen Syntaxübersetzung bereits auf Hürden gestoßen und haben uns gefragt, ob eine KI für die reine Syntaxübersetzung wirklich das richtige Werkzeug ist. In der Zwischenzeit haben wir auch andere Ansätze gesehen. Man könnte beispielsweise einen Parser entwickeln, der die Syntax anhand fester Regeln übersetzt. Die KI sehe ich mittlerweile eher auf einer höheren Flugebene als beratendes Tool, das Zusammenhänge aufzeigt und Verbesserungspotenziale im Modell identifiziert.
Limitierung_KI_Tool	